This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Date of publication xxxx 00, 0000, date of current version xxxx 00, 0000.

Digital Object Identifier 10.1109/ACCESS.2024.Doi Number

C2IDTL: Novel Click to Image Conversion Approach for Deep Transfer Learning in Click Fraud Detection on Digital Platforms

CHANNA FERNANDO[1], CHAMILA WALGAMPAYA[2] AND JANAKA ALAWATUGODA[3,4,*]

[1]Computer Center, Faculty of Engineering, University of Peradeniya, Peradeniya, Sri Lanka
[2]Department of Engineering Mathematics, Faculty of Engineering, University of Peradeniya, Peradeniya, Sri Lanka
[3]Research and Innovations Centers Division, Rabdan Academy, Abu Dhabi, United Arab Emirates
[4]Institute for Integrated and Intelligent Systems, Griffith University, Nathan, Queensland, Australia

Corresponding author: Janaka Alawatugoda (e-mail: jalawatugoda@ra.ac.ae).

ABSTRACT The global retail industry has witnessed a massive expansion of e-commerce businesses in the last decade by transforming physical stores into digitalised entities. One of the biggest threats toe-commerce is click fraud, which generates fraudulent clicks without genuine intention on online advertisements using automated software programs or organised human activities. This has become a major threat to the online advertising eco systems, causing heavy losses for leading tech companies like Google, Facebook, Meta, and Yahoo, which rely on pay per click (PPC) models. To overcome click fraud, we propose a novel click to image conversion technique to generate synthetic images from clicks and perform click fraud detection on those generated images directly with Xception, Inception, VGG, ResNet and DenseNet DTL models (C2IDTL). This novel approach embedded the feature importance and spatial structure into synthetic images to enable the identification of complex pattern recognition via DTL models. We evaluated the performance of C2IDTL on three publicly available datasets, namely FDMA BuzzCity 2012, Talking Data click dataset and Sweden Ad click datasets and compared the performances with the conventional state-of-the-art models. C2IDTL enhances the fraud detection by exceeding 91.18% accuracy, 90.90% F1-score and 96.45% AUC based on the K-fold cross validation performed on all three datasets. Consequently, these results validate the robustness and the ability to generalise across diverse click datasets evaluated in prior scientific literature. Despite DTL's association with the visual domain, this study challenges its traditional usage and opens avenues for future studies to optimise performance and robustness.

INDEX TERMS C2IDTL, Click Fraud, Deep Transfer Learning (DTL), CNN, Machine Learning

I. INTRODUCTION

E-commerce has reshaped the retail industry in the last decade [1]. The market revenue in 2016 was 1.1 trillion, and in 2016 it was 3.5 trillion, which shows more than three times growth globally [2]. This e-commerce continuous growth is fuelled by the rapid growth of the internet and mobile and computer usage [3]. Consumers can easily see advertisements for these businesses whenever they use web browsers to surf the internet. As a result, the quality and convenience of the end to end business process have improved with the advancement of technology [4]. Unfortunately, these advancements introduce new challenges for the e-commerce industry.

One such major challenge is known as click fraud [5]. This is a common digital crime in the digital world that happens in online advertisements. Usually, these ads redirect users to the advertiser's website. So when users click on an advertisement, they will automatically point to a different e-commerce business site while leaving the previous web page. Click fraud happens when a computer, a human, or an automated script mimics a genuine online customer by generating fake clicks on advertisements without having an interest in visiting the targeted advertisement link [6]. These fake clicks are a huge threat to the online advertising ecosystem [7]. This industry is a primary source of revenue for leading tech firms like Google, Yahoo, and Facebook. Their web search engines

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

generate a tremendous income by selling ad clicks generated by those advertisements. It is called the pay-per-click (PPC) business model [8]. These companies are responsible for taking action for those fraudulent clicks on their ad networks. Usually, they use different detection and prevention models and algorithms to categorise clicks as either fraudulent or genuine. Currently, most of the latest detection systems are based on machine learning and deep learning models [9]. The latest trend among them is using convolutional neural network (CNN) deep neural architectures [10].

These CNN networks can be used on multidimensional datasets to learn complex and non-linear relationships. They can easily recognise different features of the input data and find patterns in between [11]. According to new studies, CNN based deep learning techniques are being applied in a wide range of applications, including audio, voice, visual data, and natural language processing (NLP), and have shown incredible results. However, once we train such a model for a specific application, we have to reset its parameters and train it again for other applications. In such cases, transfer learning comes in handy [12]. This technique allows users to use the same trained model that was previously trained for a specific application to be used on a different task [13]. Despite CNNs being used for image analysis with success, they may not be fully utilised in modelling the capabilities of these kinds of data, as many other areas deal with non-image data [14]. In the click fraud domain, our dataset would be a 1D set of clicks generated on advertisements. But most of the transfer learning models were being trained with two dimensional input datasets. We cannot directly use 1D click datasets with such transfer learning algorithms.

DTL has demonstrated their effectiveness in image classification in various other domains including medical imaging [15], industrial fault diagnosis [16], and remote sensing [17]. However, click fraud domain still not benefited from these advancements. To bridge this gap, we propose C2IDTL. We propose a novel click to image conversion technique, to convert a 1D click dataset to a 2D image dataset and then perform click fraud detection on the generated images using DTL models. These converted images have been applied with different DTL models to evaluate the binary classification performances on the FDMA BuzzCity 2012 [18], Talking Data [19], and Sweden Ad Click [20] datasets. Finally, we have compared classification results with state-of-the-art models. Following are the main contributions to our paper.

- Proposed a novel technique C2I to transform click data into two dimensional(2D) synthetic images (Non image data to image conversion).
- Developed a preprocessing module to enhance click data before applying proposed DTL models for click fraud detection.
- Investigated different image augmentation techniques for reducing overfitting and improving performances on unseen data.
- By applying the proposed C2I novel technique, the click data was transformed into synthetic images, enabling the use of DTL models, which are specifically designed for higher dimensional input domains. Since the raw click data cannot directly fetch as input for DTL models, this transformation bridges that void thus allowing for the application of DTLs in the click fraud detection domain. Furthermore, this approach significantly lowers computational costs and time by leveraging pre trained models, thereby enhancing the fraud investigation process.
- Complete experimental analysis to assess the performances of the DTL models utilizing three well know click datasets by evaluating accuracy, precision, recall, F1 Score, and AUC Score and in terms of Computational parameters.
- Provided detailed training and validation loss variations of the DTL model in different folds and training cycles to explore model stability and convergence.
- Complete performance comparison against traditional state-of-the-art models utilized in previous literature.

The rest of this paper is organised as follows: Section II reviews the related literature on click fraud detection using DTL based techniques. Section III introduces the proposed method and its components in detail. Section IV presents the quantities and qualitative experimental results. Finally, Section V concludes the paper and suggests potential directions for future research.

II. RELATED WORK

The detection of click fraud in mobile advertising is an important field of research, as it may result in significant financial losses and distrust between advertisers and ad networks. Different deep learning models and machine learning have been suggested to tackle this problem.

A. ML BASED CLICK FRAUD DETECTION

In the past studies before ML based techniques became popular most of the studies were done using statistical based approaches. Wang et al. [21] proposed a rule based statistical model in their research. As a distinguished study, they analysed both users' end and middle traffic layers to identify click patterns. Singh and Sisodia et al. [22] focused on using a Gradient Boost model as their primary detection model in their fraud click detection work. Minastireanu et al. [23] also demonstrated fraud detection efficacy in terms of speed and memory usage with the XGBoost algorithm. Dash and Pal et al. [24] generated a new robust feature set using existing features by exploring feature engineering and the generated

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

new feature set then applied SVM, KNN and GBDT models for fraud detection. By converting quantitative features to numerical Jianyu et al. [25] applied the GDBT and XGBoost model to identify fraud click on frequency domain. Gohil et al. [26] analysed the potential of the XGBoost model by separating time stamps into month, day and hour thus creating novel features in addition to the existing feature set. Moving in another direction Thejas et al. [27] proposed a cascaded XGBoost model with a Random Forest classifier. This model demonstrated better performances compared to the conventional original XGBoost model. The open source Gradient Boost Libraries H2O and Catboost were utilized by Dekou et al. [28] to assess stacked different basic models to detect illegitimate clicks. Mikkili et al. [29] presented LR and GNB-based hybrid detection systems for invalid clicks detection. Aberathne et al. [7] propopairedsed a real time click investigator based on a hidden Markov model based ML model. This approach reduced the data training complexity of the dataset. Click fraud detection is far more difficult due to the use of distributed attacks using botnets. Kirkwood et al. [30] suggested an ML-based click fraud detection method for click fraud detection. Different machine learning algorithms, including logistic regression and neural networks, were employed in this study. Mahesh et al. [31] developed several machine learning models, and their efficiency was evaluated via a comparative performance to analyse their effectiveness on click fraud detection.

B. DL BASED CLICK FRAUD DETECTION

State-of-the-art models cannot perform well with nonlinear datasets. If the dataset has more complex nonlinear relationships between features and outputs, ML models will have a hard time figuring those out. On the other hand, we can overcome that issue by adapting DL models. Fernando et al. [10] developed an improved CNN based network to identify fake clicks, tested with the Talking Data dataset. FINet by Deepti Sisodia et al. [32] is a CNN-based system designed to detect click fraud, overcoming the scarcity of pre-trained models. Mouawi et al. [33] utilized ANN based approach as a third party crowdsourcing mechanism to control fraud clicks. C.D. Fernando et al. [10] developed an improved Lenet-5 based network to identify fake clicks. Gabryel et al. [34] proposed an FCNN based DL model to perform click fraud detection using the browser data of the users. Chari et al. [35] investigated different booting techniques with different feature engineering techniques. They were able to generate new features by combining several original features. Thejas et al. [27] suggested a combined model using ANN and AE for click fraud detection on an extremely unbalanced data set. Renström et al. [36] unsupervised learning based encoder approach as a three step approach to overcome the deficit of labeled clicks. Bartool et al. [37] also proposed a CNN based ensemble technique to apply BiLSTM and RF as a hybrid fraud detection approach. Sreekala et al. [38] developed a Deep Neural Network approach to distinguish fake and legit click by identifying click patterns. To address the struggle of the capturing multi dimensional data by traditional ML models, Joy et al. [39] proposed a tensor-based approach for click fraud detection using BiLSTM and attention-based CNN to improve upon traditional LSH tensor recovery methods. Alzahrani et al. [40] was assessed nine ML and DL models in a thorough study on click fraud detection. The results showed all the selected tree-based algorithms (DT, RF, GB, LGBM, and XGBoost) scored over 98.9% accuracy, while DL models like RNN also performed strongly. Using the Gemma-2b model with LoRA fine tuning Dai et al. [41] developed an LLM based model to improve the feature extraction.

C. NON IMAGE TO IMAGE CONVERSION

However, those DL models require huge time and computational costs for training [42]. Also, they need huge datasets for the training. As for the solution in this study, we are trying to overcome this gap by using DTL models [43]. The major issue is we cannot apply clicks as input for the DTL models. We should syntactically generate images to do so. Essentially our goal was to convert tabular data to 2D grid based images. As a non image to image conversion technique "DeepInsight" proposed by Sharma et al. [44] on converting non image datasets to generate syntactic images. This study was based on 4 main datasets which were a gene-expression dataset, a text dataset, a vowels dataset and two artificial datasets. They have successfully generated images by utilizing tSNE and PCA based dimensionality reduction algorithms. Another popular approach is IGTD (Image Generator for Tabular Data) proposed by Zhu et al. [14] They have also proposed a novel image generation technique using tabular data. In this proposed technique, they tried to assign similar pixel values for similar features and then they generated a syntactic image which represents one row of the tabular dataset. This was applied to analyse gene profiles of cancer cells. They have observed better performances in predicting cancer prevention drug responses with this generated image dataset compared to the same analysis done with original tabular data. "DeepFeature" by Sharma et al. [45] was also developed to convert non image data to images to apply deep CNN based models. Here they applied the "DeepFeature" pipeline to perform gene selection as an improvement on the original "DeepInsight" pipeline. Results showed that this pipeline also performed gene selection better compared to the original "DeepInsight" pipeline. These conversion techniques demonstrate that the potential of applying deep CNN based models on non image datasets by converting them into images. In this study, our goal is also to convert non image clicks using our proposed novel technique CTIDTL on publically available click datasets and evaluate performance by comparing them with state of art ML models.

D. LIMITATION OF THE LITERATURE

Very few amount of literature has been done on using deep neural networks to transform non-image data into images on

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

the click fraud domain. Sisodia et al. [22] proposed DCNNTr (deep convolution neural network-based transfer learning) to utilize deep neural networks as the primary feature selection technique. Then the selected feature set applied a modified tSNE dimensionality reduction algorithm and performed the non image to image conversion on the FDMA BuzzCity 2012 dataset. But in our approach first, we apply proposed pre engineering techniques on all the click datasets (FDMA BuzzCity 2012 dataset, Talking Data dataset, and Sweden Ad Click datasets) and then we generated images by applying our proposed novel Click to Image (CTI) technique. Then we utilized selected pre trained DTL models on the generated image dataset and performed click fraud detection by directly applying the syntactic image dataset generated in the above step with the least amount of data preprocessing. In DCNNTr, click fraud detection was performed by using five baseline ML models. But in our case, we utilized pre trained DTL models (Xception [46], InceptionV3 [47], VGG16 [48], ResNet50 [49] and DenseNet121 [50]) on the click classification due to the better performance of these pre-trained DTL models compared to ML models.

III. METHODOLOGY

In this study, we propose an innovative novel approach C2IDL to transform one dimensional clicks into images which are then applied to perform click fraud detection as a two dimensional structured inputs classification with different DTL models for click fraud detection. We have tested this novel approach with FDMA BuzzCity 2012, Talking Data, and Sweden Ad Click datasets to evaluate this approach since most of the previous studies were also tested and evaluated with these datasets.

A. DATASETS

We have selected three publically available datasets for our study which have been extensively cited in many studies. These three datasets are the most used and publically available as click fraud datasets. These click data were collected from digital advertising platforms and the studies done on top of them were also focused on detecting fraud clicks. Also, these datasets share similar technical details related to the clicks such as timestamp, device Id, IP etc. Indicating the similarity among them and the possibility of applying similar preprocessing techniques. Consequently, we applied uniform data preprocessing techniques for these selected data samples. Also, these clicks were labelled clearly indicating whether they were fraudulent or legitimate. For FDMA BuzzCity 2012, we applied similar preprocessing techniques detailed in Taneja et al. [51], Perera et al. [52] and Oentaryo et al. [53]. For Talking Data Ad click dataset, we followed the techniques applied by, Fernando et al. [10], Sisodia et al. [32] and Batool et al. [37]. Finally, for the Sweden Ad Click dataset, the preprocessing was guided by Aberathne et al. [7].

These selected datasets are the most cited in the domain of click fraud detection and are available publicly to inspire the re-applicability in novel studies. Even though most of the above cited studies were done on a single dataset, this work provides a more comprehensive analysis on click fraud dynamics by employing multiple datasets.

In this section, we are going to discuss each dataset with the input features and the output features rigorously. In real world scenarios, most of the clicks are illegitimate and generate fewer amount of legit clicks. The selected datasets also show a high imbalance towards the illegitimate click class thus validating the usage of a real world click fraud detection application offering a comprehensive representation.

1) FDMA BUZZCITY 2012

The global mobile network company BuzzCity Pte. Ltd. held a competition primarily to analyse clicks made on their mobile advertisements between the second week of 2012 February and the second week of 2012 March and to develop fraud detection technologies [18]. They collected data over three days during the mentioned period. Table 1 shows a summary of the dataset.

TABLE I
SUMMARY OF THE FDMA 2012 DATASET

			No of publishers			
	Duration	Clicks	Fraud	Observation	OK	Total Clicks
Train	9 – 11 Feb 2012	3173834	72	80	2929	3081
Validation	23 – 23 Feb 2012	2689005	85	84	2895	3064
`Test	8 – 10 Mar 2012	2598815	82	71	2847	3000

Since then, numerous studies have been done on this dataset using different techniques to differentiate between fraudulent and authentic clicks. It primarily contains the ad publisher's profile details and click records made on those publishers' advertisements. The publisher records contain details related to the ad publisher, such as bank account, ID, address, and status. Table 2 presents a detailed description of those features. The output of the data set is the status field, and it has three statuses.

- OK: this indicates that this publisher was sending legit traffic.
- Observation: The publisher's traffic was not clearly identified as legitimate or fake. This was mainly due to either the publisher being a newer one or Buzz City data analysts couldn't clearly conclude its validity.
- Fraud: The publisher was clearly identified as sending malicious traffic to the system.

TABLE II
FDMA 2012 DATASET - FEATURES OF THE PUBLISHER'S DATASET

Attribute	Description
address	Mailing address of a publisher

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

publisherid	Unique identifier of a publisher.
bankaccount address	Bank account associated with a publisher
status	OK: Publishers whom BuzzCity confirms as having healthy traffic (or those who cleared from existing fraud detection systems). Observation: BuzzCity does not have any conclusive agreements with these publishers. (Most probably those who have just started sending data traffic to the system or publishers who may have just started their traffic or whose traffic deviates from other traffic patterns.) Fraud: Publishers who are confirmed as fraudulent with solid proof. BuzzCity suspends their accounts. (They won't be paid anymore.)

Then the click records contain details regarding the clicks, such as the IP address, click ID, device ID, and time of the click, and Table 3 presents a comprehensive version of the details related to clicks. These details are not the real details. These data were anonymized so that the privacy of their user details in the dataset is protected. The process makes sure that sensitive information is preserved confidentially while overcoming the risk of privacy breaches and unauthorised access to sensitive information. It's mandatory to protect such data to ensure individuals' privacy rights and follow data protection regulations.

TABLE III
FDMA 2012 DATASET - FEATURES OF THE CLICK DATASET

Attribute	Description
id	Unique identifier of a particular click.
numericip	Public IP address of the click origin.
deviceua	Phone model of the click
publisherid	Publisher's unique identification
campaignid	Unique identification of the ad campaign
usercountry	Country of the user click.
clicktime	Timestamp of the click.
referredurl	URL where ads are clicked
channel	co: Community. es: Entertainment and lifestyle. gd: Glamour and dating. in: Information mc: Mobile content. pp: Premium portal. se: Search, portal, services.

2) TALKING DATA DATASET

This dataset is also freely available in the Kaggle community. It was provided by Talking Data [19], which is one of the top-notch data intelligent service providers. They held a competition to develop an algorithm to predict whether their users would download the mobile app mentioned in this particular advertisement.

The dataset was gathered over a span of 4 days and contains more than 200 million clicks. It contains click related features such as the IP address, application ID, and device ID. Table 4 illustrates a detailed version of the click features. These data were also anonymized so that the privacy of their user details in the dataset was protected. The output feature of the dataset is 'is attributed,' which has two possibilities:

- 1: defines a legit click, and a user downloads the mobile application when s/he clicks on the advertisement.
- 0: a malicious click, and the user did not install the mobile application after clicking on the advertisement.

TABLE IV
FEATURES OF THE TALKING DATA DATASET

Attribute	Description
app	App id for marketing.
channel	Channel id of mobile ad publisher.
os	OS version id of user
click_time	timestamp of click (UTC)
ip	Ip address of click.
device	Device type id
attributed_time	Timestamp, if user download the app after clicking an ad

3) SWEDEN AD CLICK DATASET

This dataset was originally reviewed by Haider et al. [20]. The dataset was provided by a mobile advertising company that ranked among the top 5 fastest companies in Sweden. This European mobile ad server company received approximately 5 million click impressions daily. The data was collected over a span of 7 days and yielded a total of 260,678 records. Table 5 contains diverse data related to the data transactions that happened among different sites. It shows all the features of the delivery dataset, and deliveryId, timestamp, clientIP, UID, accountId, siteId, ipCountryCode, cookie, isImpression, and impressionTime are some of them. This event table contains the details of the generated events due to the user activities that happened on the mobile advertisement. Such as eventId, eventTimestamp, eventType, eventValue, and deliveriId.

TABLE V
FEATURES OF THE SWEDEN AD CLICK DATASET

Attribute	Description
deliveryId	platformid
timestamp	deviceModelId

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

valid	platformTypeId
clientip	browserId
adRelType	browserVersion
noAdReasonCode	ipCountryCode
forcedAd	ipRegion
testAd	ipCity
uid	ipIsp
dnt	ids
marketId	connectionType
operatorId	mcc
surfSessionId	deviceModel
uidCreationTimestamp	adSpaceReqUrl
adSpaceId	cookie
accounted	privateMode
siteid	id
clientId	isImpression
clientVersionId	impressionType
webViewSdkVersion	impressionTime
unknownDeviceId	status

B. DATA PREPROCESSING

From each dataset, we randomly selected 100,000 data samples. We then applied feature engineering techniques to transform each textual feature into an integer-based value, facilitating easier analysis and processing.

From each dataset, we randomly selected 100,000 data samples. We then applied feature engineering techniques to transform each textual feature into an integer-based value, facilitating easier analysis and processing.

For each dataset, we started data preprocessing by removing feature columns that have extensive NAN values. This step ensures the dataset's integrity by eliminating incomplete data. Then we removed any records that still contained NAN values after the initial column removal. We transformed all the remaining non-integer columns into integers. This conversion process generalised the datasets for more effective analysis. Then, in each dataset, the 'time' column was divided into separate components: year, month, day, hour, minute, and second. This granular approach allows for more detailed temporal analysis. Once the input feature preprocessing was done, the final output of each dataset was preprocessed as follows.

- For the FDMA 2012, we applied the best preformed methodology introduced by Kasun et al. [54]. In this approach, all instances labelled 'Observation' in the 'status' column were replaced with 'Ok'. This adjustment simplifies the classification by reducing the number of unique labels. Finally, the 'status' feature was mapped with 'Ok' assigned a value of 1 and 'Fraud' assigned a value of 0.
- In the Talking Data dataset, the output feature ‘is attributed’ was defined as fake clicks by value 0 and legit clicks by value 1.

Similarly, for the Sweden Ad Click dataset, the output column 'status’ denoted fraudulent and valid clicks by value 0 and value 1.

Once the initial data preprocessing was done, to address any class imbalance in the datasets, we applied the Synthetic Minority Over Sampling Technique (SMOTE) [55]. This technique generates synthetic samples for the minority class, promoting balanced class distributions and improving model performance. We applied the ‘MinMaxScaler’ [56] to normalize [57] the datasets. This scaler transforms features by scaling them to a given range, typically [0, 1], ensuring that all features contribute equally to the model training process. With that, we mapped each input feature and obtained normalised datasets. After these preprocessing steps, we obtained clean, balanced, and normalised datasets ready for further analysis of the C2IDTL. Table 6 shows the imbalance ratio of the initial input dataset and the Table 7 shows the statistics of the final processed sample datasets.

TABLE VI
IMBALANCE RATIOS OF THE INPUT DATASETS

Dataset	Imbalance Ratio
FDMA 2012	23.91
Talking Data	589.66
Sweden Ad Click	26.03

TABLE VII
STATISTICS OF THE PREPROCESSED DATASETS

Dataset	Class 0	Class 1	Total
FDMA 2012	50157	49843	100000
Talking Data	67925	67844	135769
Sweden Ad Click	41995	42005	84000

Figure 1 illustrates the box plot representation of the numerical features after applying feature engineering techniques on the FDMA 2012 click dataset. ‘Iplong’, ‘day’, ‘hour’, ‘minute’ and ‘second’ shows a moderately evenly distribution with a median of central and very few or zero extreme outliers in their distribution. ‘Agent’, ‘partnerId’, ‘cld’ and ‘cntr’ show a skewed distribution with a huge number of outliers indicating a small portion of data has higher values than the majority. ‘year’ and ‘month’ contain constant values. Overall, the FDMA dataset contains features with normal distribution and features with highly skewed distribution.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

Figure 2 demonstrates that the 'ip', 'channel', and 'second' of the Talking data dataset indicate a balanced distribution with a centered median falling between upper and lower quartiles and very low data skewness. A comparatively skewed distribution can be seen with 'app', 'device', 'os' and 'minute' features, indicating outliers with higher values. This indicates the huge portion of observation is fairly low compared to the other observations. 'year', 'month', 'day', and 'hour' show a constant distribution. With that, we can state that this Talking data dataset has a combination of skewed and balanced distribution features in this selected sample.

Figure 3 shows the click data distribution of the Sweden Ad click dataset. Here 'deliveryid', 'timestamp', 'uid', 'surfSessionid', 'adSpaceId', 'browserId', 'clientId', 'browserVersion', 'impressionType' and 'impressionTime' these features exhibit a normal distribution with a centered median indicating very few outliers. 'clientIp', 'accountId', 'osId', 'platformId', 'ipMarketId', 'ipRegion', 'ipCity', 'ipIsp' and 'cookie' box plots illustrate a highly skewed distribution with a large number of outliers. As in the Talking data dataset, most of the observations fall close to the bottom and the majority of outliers show higher values. The other features like 'adRelType', 'forcedAd', 'valid', 'platformTypeId' and 'ipCountryCod' indicate a flat distribution showing that these features are constants. By closely analysing, we can conclude that all three datasets contain a mix of skewed and balanced features, along with some variables that appear to have low to no variation. Figure 4 shows a summary of the balanced datasets after applying preprocessing techniques and SMOTE class balancing technique.

C. C2I: CLICK TO IMAGE CONVERSION

In this novel C2IDTL, we converted the finalised preprocessed click datasets to image datasets by using the novel techniques mentioned below. In this section, we briefly discuss the transformation of a non-image sample to its image form.

This novel image conversion technique allowed us to convert click stream into visual images and then apply CNN based DTL models for fraud detection. Essentially, CNNs can automatically extract unseen features [58] ,which are difficult to identify in a manual feature engineering process. We performed a Grad-CAM analysis in the result section and highlighted how the CNN networks focus on different areas (features) of the generated images. To support that, we performed a Grad-CAM analysis and presented it in the results section, and discussed how the CNN networks focus on different areas (features) of the generated images for the classification. Also, they are capable of capturing non-linear patterns comparatively better than conventional ML models [59]. The data augmentation techniques like rotation, flipping, and zooming [60] are also possible to apply to the image input dataset. This technique leads to improved classification performance, reduces overfitting and improves the model's robustness. Finally, the adaptation of pre trained CNNs enables DTL models to operate more efficiently, fostering the rapid scalability and real-time adaptation capability [61].

Figure 5 illustrates a brief summary of the C2I conversion process. Each click includes different feature sets like IP, timestamp, etc. Then these feature vectors are reshaped into small 2D matrices. To cater to the DTL models' input size requirement, these generated matrices were up-scaled depending on their feature importance. Finally, we created a standard RGB image (224 × 224) with three channels from these up-scaled matrices. Algorithm 1 shows the detailed version of the C2I process with respective mathematical representations.

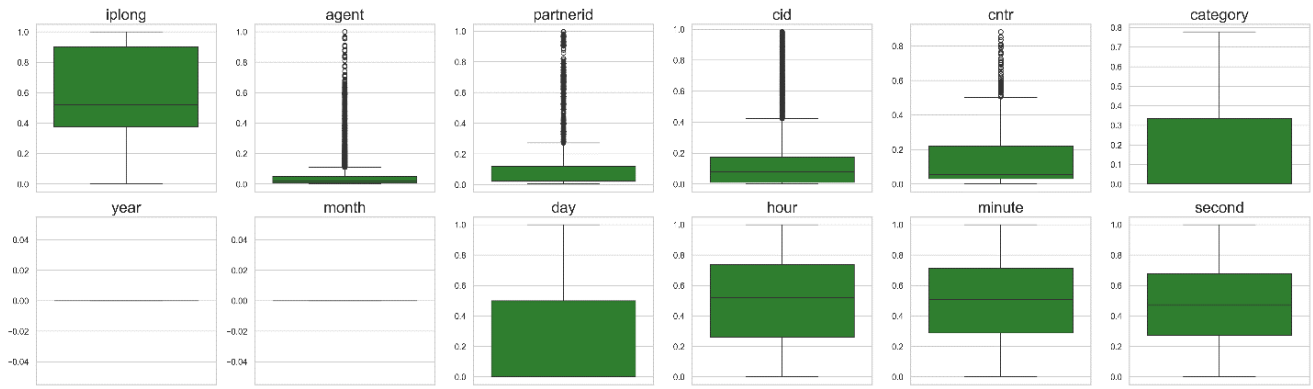


FIGURE 1. **Box plots of the FDMA 2012 click dataset**

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

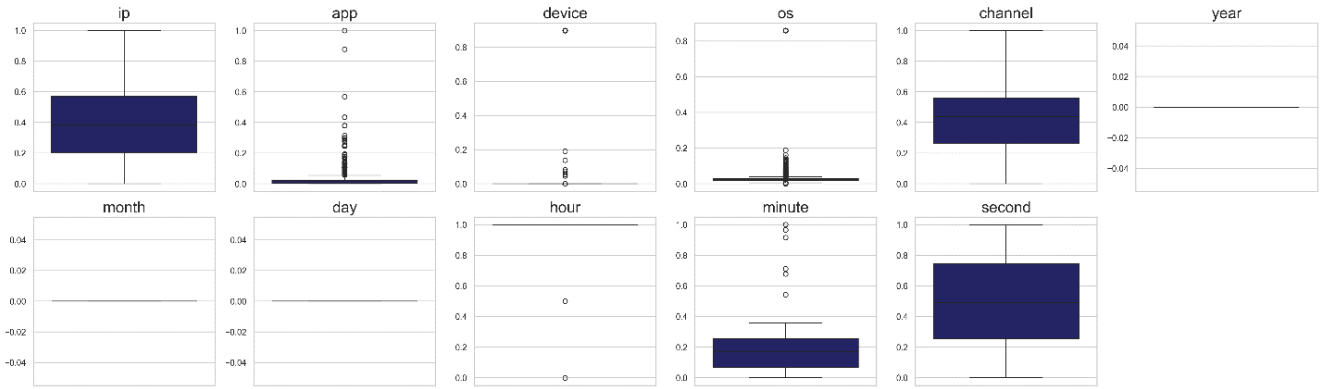


FIGURE 2. **Box plots of the Talking data click dataset**

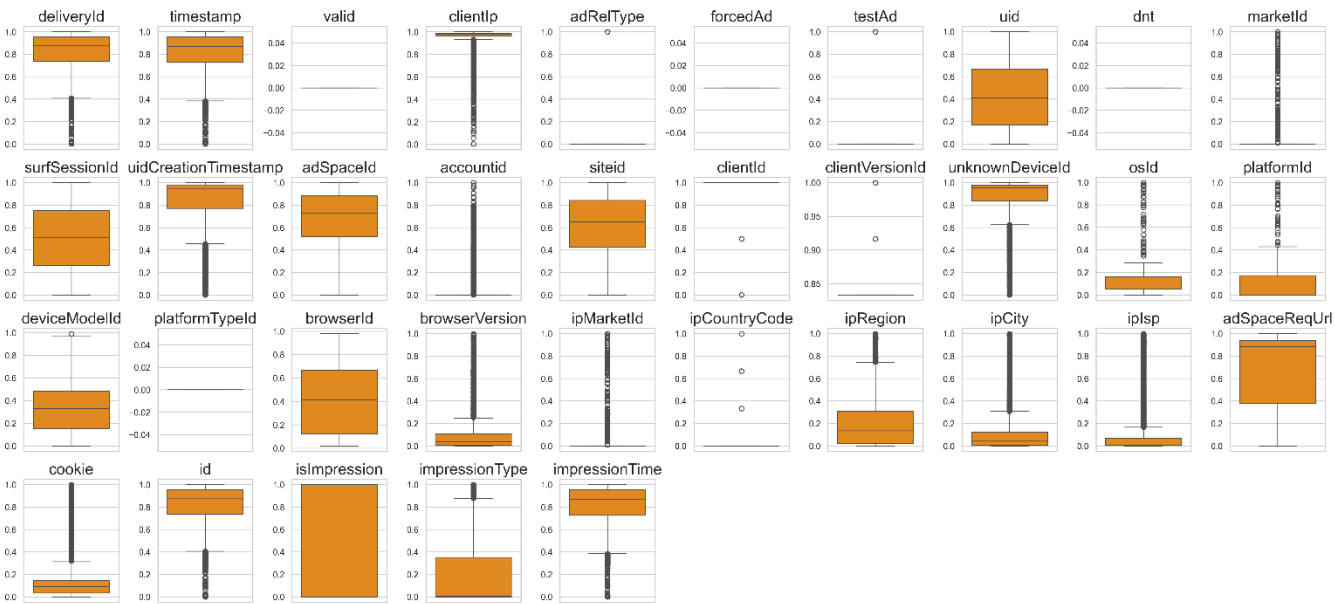


FIGURE 3. **Box plots of the Sweden Ad click dataset**

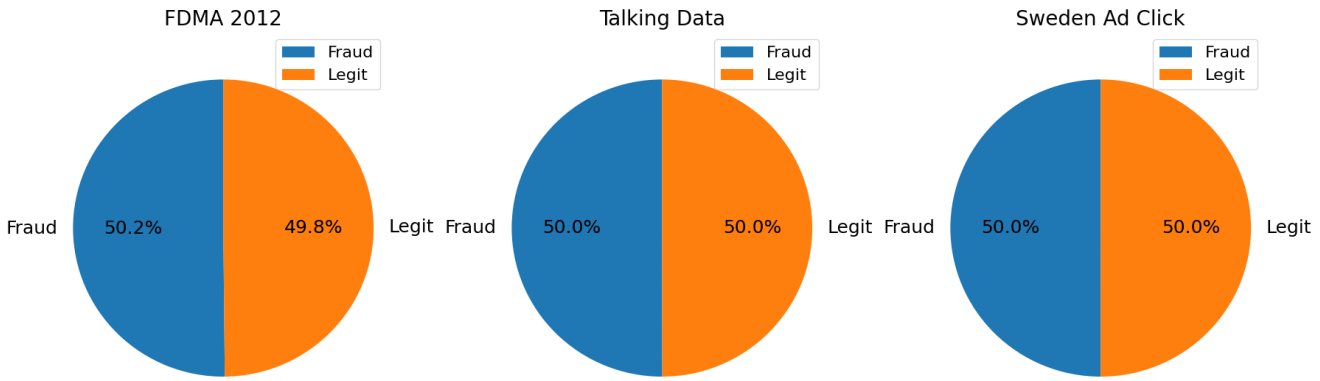


FIGURE 4. **Summary of the outputs of the balanced datasets**

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

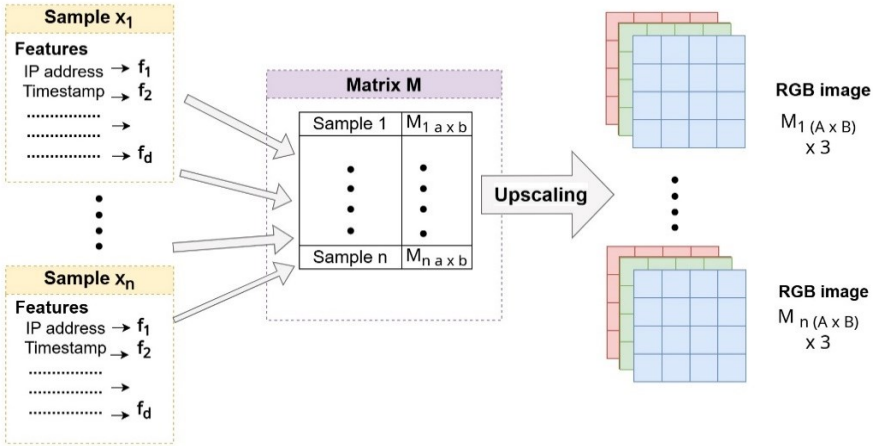


FIGURE 5. **Summary of the of Click-to-Image (C2I) process**

Algorithm 1 CTI: Convert Click to Image Datasets

INPUT:
X: A set of n samples where each sample has d features
d: Number of features in each sample
a, b: Initial matrix dimensions
A, B: Target image dimensions
$M_{a \times b}$: Initial feature matrix
FI: Feature importance vector calculated using the Random Forest algorithm

OUTPUT:
$RGB_{A \times B}$: Up scaled feature matrix transformed into an image

```
Start:
    # Define dataset as a matrix
    for each sample x_i in X do:
        X_i (1 × d) = [fi1, fi2, ..., ..., fid]
    end for
    # Generate a × b matrices for each sample in the X:
    Index = 0;
    X_initial_images = empty list of size n;
    for each Xi in X do:
        M (a × b) = zeros(a, b)
        for j from 0 to a − 1 do:
            for k from 0 to b − 1 do:
                M a × b [j][k] = X_i [index]
                index = index + 1
            end for
        end for
        X_initial_images[i] = M a × b
    end for
    # Calculate FI using Random Forest algorithm
    FI = [i1%, i2%, i3%, ... , id%]
    # Determine pixel allocation
    total_pixels = A × B
    pixel_count = 0
    # Up sampling process. This is essential because original dimensionality a × b if not sufficient to apply these generated images for DTL model.
    for each matrix Mi (a × b ) in X_initial_images do:
        M (A × B) = zeros(A, B, 1)
        pixel_count = 0
        # Up sample the dimensionality of a × b matrix to A × B
        for each feature fi in the matrix Mi (a × b) do:
            amount_of_pixels_be_replaced = round(FI [i]% ∗ total_pixels)
            #Replace zeros in M (A × B) with feature value fi
            for p = pixel_count to (pixel_count + amount_of_pixels_be_replaced − 1) do:
                M (A × B) [p] = fi
            end for
            pixel_count = pixel_count + amount_of_pixels_be_replaced
        end for
        # Create an RGB image by duplicating the M(A × B) matrix across three color channels
        RGB_image = zeros(A, B, 3)
        for i = 1 to A do:
            for j = 1 to B do:
                RGB_image[i][j][0] = M (A × B) [i][j] # Red channel
                RGB_image[i][j][1] = M (A × B) [i][j] # Green channel
                RGB_image[i][j][2] = M (A × B) [i][j] # Blue channel
            end for
        end for
        RGB[i] = RGB_image
    end for
End
```

If the training set consisting of n samples is defined as equation 1, where a feature vector has d features or $x \in \mathbb{R}^d$.

$$\chi = \{x1, x2, \dots xi, \dots, xn\} \tag{1}$$

Let's take the sample xi from the χ and denote as follows. Then we can denote its feature values as $fi1, fi2 \dots, fid$; where $f \in \mathbb{R}$. (This is a $1 \times d$ row metrics.)

$$xi = fi1, fi2, \dots, \dots, fid \tag{2}$$

Now we can define a 2D array of size $a \times b$ by converting 1D row matrix xi as follows.

$$M_{a \times b} = \begin{bmatrix} fi1 \dots & \dots fia \\ fib & fi(b+a) \end{bmatrix} \tag{3}$$

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

The problem with representing these click fraud datasets was that they had a limited number of features. For instance, the Sweden Ad Click dataset had about 50 features, making the dimension of the generated matrix very low. To apply a DTL model, we had to upscale the input image size to 224×224. To increase this small generated matrix size, we were required to add paddings (either 0s or 1s) to set the input size to match 224×224. Since this padding didn't represent the features of the click datasets, we used the following steps to upscale the generated tiny images. Let the required image size be denoted as $A \times B$ and the original small matrix size is $a \times b$. We need to transform this small matrix $M_{a \times b}$ in to a larger $M_{A \times B}$ matrix. To upscale the small $M_{a \times b}$ matrix to the required $M_{A \times B}$ size, we avoided adding just padding that didn't contain valuable information. Instead, we duplicated pixels by calculating the feature importance of each feature and determining the weighted number of pixels to be represented by each feature in the small $M_{a \times b}$ matrix. We calculated the feature importance using the Random Forest algorithm [62]. Let the percentage of each feature's importance be denoted as $i_1\%, i_2\%, i_3\%, \dots\dots, i_d\%$ where $i \in \mathbb{R}$. Now the feature matrix FI is;

$$FI = [i_1\%,\ i_2\%, i_3\%, \dots\dots, i_d\%] \tag{4}$$

The final $A \times B$ matrix should include $i_1\%$ of $fi1$ pixels, $i_2\%$ of $fi2$ pixels, … …, $i_d\%$ of fid pixels. Mathematically, this can be expressed as:

$$\sum_{j=1}^{d} (i_j\% * A \times B) = A \times B \tag{5}$$

Thus, the $a \times b$ matrix of the ith sample is up sampled to an $A \times B$ matrix by duplicating features according to their feature importance:

$$M_{A \times B} = UpSample\ (M_{a \times b}, FI) \tag{6}$$

This approach ensured that the up scaled image retained valuable information about the dataset, rather than relying on artificial padding that lacked contextual relevance. Now we generated image datasets by applying the above mentioned techniques and obtained three sets of image datasets from each dataset. These image datasets contained the same number of images as in Table 7. Now we applied these generated images as the input for the DTL models for C2IDTL.

D. DEEP TRANSFER LEARNING

Training CNN models has become challenging due to the lack of huge, labelled datasets. So it is really difficult to train a CNN model from scratch. This is why transfer learning comes in handy. They have been well trained on huge datasets, such as ImageNet [63]. This enables typical CNN networks to see unseen data from different image datasets and tune their internal parameters. This leads them to overcome more non-linearity and outlier handling capabilities. Hence, most of the image analysis tasks consist of analysing straight lines and the curves within the image; these pre trained weights could be kept as they were for our studies as well [64]. Not only that, those models may have trained on clusters of GPU and TPU resources and might have taken several weeks to fully train on the datasets. So we can save a lot of computational cost by adapting DTLs in our sturdy. We are going to apply our generated 3 image datasets for the FDMA 2012, Talking Data dataset, and Sweden Ad Click dataset as follows. As for the DTLs, we used Xception [46], InceptionV3 [47], VGG16 [48], ResNet50 [49] and DenseNet121 [50] pre trained DTL models in our study.

1) DATA AUGMENTATION

The common issue with CNN is that they perform well with the training dataset, but performance drastically decreases with other datasets. That is overfitting. To overcome overfitting in our datasets, we had to apply augmentation techniques [65] before applying the images to the DTL models. Then we applied random zooming, shearing, brightness changing, rotations, axial flipping, and other augmentation techniques as mentioned following sections.

2) DENSE LAYER

A dense layer also referred to as a Fully Connected Layer is also a neural network component which consists of any number of neurons all of which are connected with the previous layer [66]. Usually, they are at the end of the CNN networks and map extracted features into a classification possible format.

3) DROPOUT

This is also a technique to overcome the overfitting issues in the DTLs. Here, some of the neurons are purposely ignored in the classification task. Since we are removing some neurons from the network, they cannot rely on any single feature [67]. They have to learn complex, nonlinear relations between inputs. Dropout shows great results for the overfitting issues.

4) ADAMS OPTIMIZER

DTL experts widely employ the Adams optimizer [68] for its capability of adjusting learning rates adaptively, which makes it ideal for meeting different layers' changing learning demands. By employing gradient first and second moment estimates, it is possible to reduce computational time, thus training fast even when using large datasets. Sisodia et al.[32] in their study has observed the Adams optimizer converges well compared to the other optimizers for these click fraud datasets.

$$\theta_t = \theta_{t-1} - \eta\, \widehat{m_t} / \sqrt{\widehat{v_t} + \varepsilon} \tag{7}$$

Here in the equation 7, θ_t is the updated model parameters at the time stamp t and θ_{t-1} is for timestamp $t-1$. η is the

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

learning rate. $\widehat{m_t}$ is the bias corrected at the first moment estimate which is the moving average of the gradients. $\widehat{v_t}$ is the bias corrected for the second moment estimate which is the moving average of squared gradients and the ϵ is the small constant.

5) PROGRESSIVE NEURAL NETWORKS (PNNS)

Progressive Neural Network is also a technique which can be applied on DTLs as a preserving past leanings of a network with itself and using in a different task by preventing training the network from the scratch. This approach helps to make decisions while keeping previous knowledge by adding new layers on top of the previously trained model rather than updating or modifying existing layers [69]. This makes PNNs retain previous learnings and grab new learnings into newly added layers.

E. OUR APPROACH (C2IDTL: CLICK TO IMAGE DEEP TRANSFER LEARNING) WITH HYPER PARAMETER TUNING

Figure 6 shows a summary of the proposed C2IDTL system. First, the generated input image size was set to 224 * 224 RGB to ensure the compatibility of the standard input for DTL models. In an attempt to overcome common overfitting issues and the robustness of CNN-based models, different image augmentation techniques were used. First, the generated images were normalised by a factor of 1/255. This was to make sure all the image pixel values remained between 0 and 1. Then the second image augmentation shearing was applied within the 0.2 range. This small range and shearing caused small distortions to the images to withstand model performance for slight input data variations. Similarly, we then applied a zooming range of 0.2 and horizontal flipping to improve the robustness of the input data changes and overcome the overfitting of DTL models.

The DTL models were already pre trained on the 'imagenet' dataset. These models were employed in different studies and have shown incredible results. This dataset contained different images belonging to more than a thousand classes. With that, these DTL models have already learned valuable features, and we can use those pre trained values to detect fraud. As for the pooling, average pooling [70] was used to reduce the spatial dimensions of the feature maps while still preserving important features, resulting in the prevention of overfitting. Following the feature extraction, a flattened layer was used to convert the pooled feature maps into a one dimensional vector, making them compatible for adapting the fully connected layers.

The we froze all the layers within the selected base DTL models and modified each network as a Progressive Neural Network (PNN). We kept all the layers and parameters within DTL models unchanged and added following new layers on top of them. But we had to remove the final dense layer from each network since they were originally trained for classifying inputs into 1000 classes. Also, we updated the input to support 224 × 224 × 3 images which were generated via the previously explained novel C2I approach.

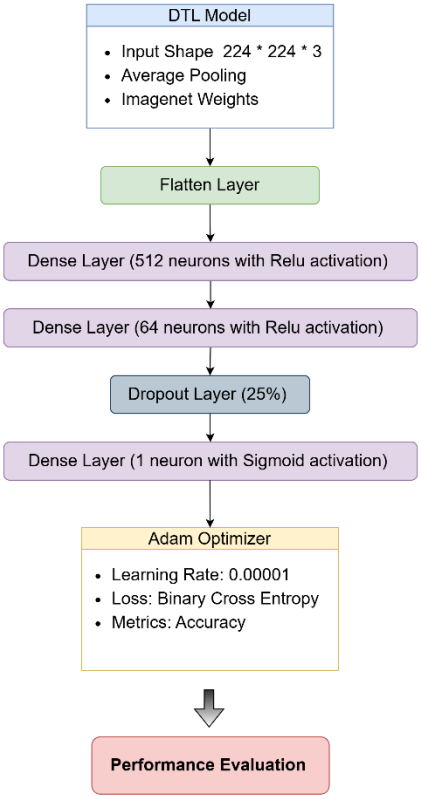


FIGURE 6. Summary of the proposed C2IDTL click fraud detection system`

A dense layer with 512 neurons and ReLU activation [41] was then applied to introduce non-linearity and enable the network to learn complex patterns in the data with another layer followed by 64 neurons with ReLU activation. As another technique to combat overfitting [71], a dropout layer with a rate of 0.25 was included, randomly disabling 25% of the neurons during training, which forces the network to learn more robust features.

Finally, to perform binary classification (identify fraud and genuine clicks), a dense layer that contained a single neuron and sigmoid activation was used. As for the optimizers, we utilized the conventional Adam optimizer. The Conventional Adam optimizer was selected due to its capability to overcome sparse gradients efficiently. Its learning rate was set to 0.00001 ensure stable and incremental convergence during the data training phase.

We used the k-fold cross-validation [72] technique to validate DTL performances. Here we set the k value to 3 to ensure the computational efficiency and calculation robustness [73], [74], [75], [76], [77]. Based on preliminary test results, increasing the fold count to 5 or 10 did not yield significant performance improvements for all three datasets. Since the primary objective of our study was to test the performance of the DTL models on the generated

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

click image datasets, subsequently, the experiment was done with k=3. As mentioned above, the higher fold values did not yield notable performance gains. Moreover, due to the computational burden for utilising resources for this study, we selected 3-fold cross-validation. Note that after each fold, we reset the model weights to their initial weights. This method of saving and loading the initial weights in each fold helps us implement and validate the model. Then the model performances were measured in each fold, and we calculated the average for each fold and presented the final value as the DTLs performance indicators.

F. STATE OF THE ART MODELS

As in the DTLs model performance evaluations, we used k-fold cross-validation on the same dataset, which was used to generate images for each dataset. These non-image data applied to traditional k-nearest neighbors, gradient boosting trees, support vector machines, and neural networks (MLP) evaluated the models performance by calculating average statistical parameters for the comparison.

G. HARDWARE AND SOFTWARE SETUP

All the results presented in the next section were obtained by a server with Intel® Xeon Gold CPU of 2.10 GHz with 32 cores processor with 128GB RAM. As for the image processing libraries, we used TensorFlow [78] and other supportive Python libraries. Also, the GPU we utilized was an NVIDIA Tesla [79] P40 with 24GB memory.

H. STATISTICAL AND COMPUTATIONAL METRICS

To assess the machine learning model comprehensively we have to analyse both statistical and computational performances. As for statistical parameters we have evaluated accuracy, precision, recall, and F1 score as a comparative analysis to have a clear insight on the different DTL model's predictive abilities. Also, inference time and memory consumption and Grad CAM analysis to check the ability to test the practical usability in the real word scenario.

In the following statistical equations, we have used the following abbreviations for TP as True Positives, TN as True Negatives, FP as False Positives and FN as False Negatives.

1) ACCURACY

$$\frac{TP + TN}{TP + TN + FP + FN} \tag{9}$$

2) PRECISION

$$\frac{TP}{TP + FP} \tag{10}$$

3) RECALL

$$\frac{TP}{TP + FN} \tag{11}$$

4) F1 SCORE

$$\frac{2 * Precision * Recall}{Precision + Recall} \tag{12}$$

5) AUC

$$\int_0^1 TPR(FDR)d(FPR) \tag{13}$$

This is the area between TP Rate vs. FP Rate which is the area under the ROC curve.

6) INFERENCE TIME AND TRAINING TIME

The training time is the total time taken to train a model using the training dataset. The amount of time taken to make a new prediction on a dataset by a trained machine learning model is the inference time.

7) MEMORY USAGE

This is the total memory consumption by the model training phase. This is calculated in MBs in our analysis.

8) GRADIENT-WEIGHTED (GARD-CAM) CLASS ACTIVATION MAPPING

GRAD-CAM is a visualizing technique which has been specifically developed to get an idea about how the deep learning model makes its predictions [80]. These GRAD-CAM images highlight the areas where the model is more concerned when a prediction is made.

$$\boldsymbol{Gradient = \frac{\partial y^c}{\partial A_{ij}^k}} \tag{12}$$

The gradient computation is done by calculating the gradient of the output class $\boldsymbol{C}$ $\boldsymbol{y^c}$ respect to the activations $\boldsymbol{A_{ij}^k}$ of the finale convolution layer. This quantifies the impact on the predicted score by each element $(\boldsymbol{i}, \boldsymbol{j})$ in the feature map $\boldsymbol{k}$.

$$\boldsymbol{\alpha_k^c = \frac{1}{Z}\sum_i \sum_j \frac{\partial y^c}{\partial A_{ij}^k}} \tag{13}$$

Here this $\boldsymbol{\alpha_k^c}$ parameter is calculated by taking the mean gradient over the feature space $(\boldsymbol{i}, \boldsymbol{j})$. Z is the total number of feature elements in the feature space which is $\boldsymbol{i} \times \boldsymbol{j}$. Hence in the context of $\boldsymbol{\alpha_k^c}$ indicates the importance of the feature map $\boldsymbol{k}$ for predicting class $\boldsymbol{C}$.

$$\boldsymbol{L_{Grad-CAM}^c = Relu(\sum_k \alpha_k^c A^k)} \tag{14}$$

Finally, $\boldsymbol{L_{Grad-CAM}^c}$ is calculated by taking the average weighted sum of the feature maps $\boldsymbol{A^k}$ with the multiplication of the calculated $\boldsymbol{\alpha_k^c}$ and by finally applying the Relu function.

IV. RESULTS

In this section, we presented our results as follows. First, we applied the C2I conversion techniques on the samples of each dataset and showed generated syntactic images using each

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

sample. In the second part, we showed the output images when the selected augmentation techniques were applied. These augmentations help to overcome overfitting in the training phase. In the final section, we showed the results of the calculated statistical and computational parameters comparatively with each selected DTL model and state-of-the-art models.

A. IMAGE GENERATION

In C2IDTL, the proposed click-to-image conversion techniques were evaluated on three well-known distinct datasets, namely the FDMA 2012 dataset, the Talking Data dataset, and the Sweden Ad Click dataset. Figure 10 shows the generated click images of classes 0 and 1 by using the proposed novel click to image conversion techniques with the FDMA 2012 dataset and Figure 7 shows the original click data before conversion.

	agent	partnerid	cid	cntr	category	year	month	day	hour	minute	second
1	0.029477	0.394263	0.006261	0.425641	0.444444	0.0	0.0	1.0	0.434783	0.847458	0.949153
2	0.082684	0.621081	0.632407	0.041026	0.444444	0.0	0.0	1.0	0.565217	0.203390	0.898305
3	0.048572	0.745163	0.085675	0.107692	0.777778	0.0	0.0	0.0	0.782609	0.542373	0.186441
4	0.059696	0.021014	0.152718	0.220513	0.000000	0.0	0.0	0.0	0.130435	0.186441	0.152542
5	0.048572	0.745163	0.108888	0.107692	0.777778	0.0	0.0	0.0	0.782609	0.491525	0.915254
6	0.012236	0.021014	0.010690	0.025641	0.000000	0.0	0.0	1.0	0.043478	0.898305	0.898305
7	0.001112	0.565043	0.166158	0.030769	0.333333	0.0	0.0	1.0	0.826087	0.237288	0.152542
8	0.006674	0.021014	0.015119	0.179487	0.000000	0.0	0.0	0.0	0.391304	0.847458	0.084746
9	0.006674	0.121748	0.154551	0.220513	0.000000	0.0	0.0	0.0	0.695652	0.288136	0.000000
10	0.010938	0.021014	0.010690	0.087179	0.000000	0.0	0.0	1.0	0.086957	0.457627	0.745763

FIGURE 7. Original click data sample (FDMA 2012 dataset)

Using the same novel conversion technique, we randomly selected 10 images from the Talking Data dataset, as illustrated in Figure 11. It is clear that both classes can be easily distinguished just by looking at them briefly. Anyone can do that without using any data processing techniques and Figure 8 shows the original data sample.

	app	device	os	channel	year	month	day	hour	minute	second
1	0.014754	0.000296	0.011315	0.265182	0.0	0.0	0.0	1.0	0.135593	0.762712
2	0.029508	0.000296	0.026874	0.238866	0.0	0.0	0.0	1.0	0.101695	0.966102
3	0.039344	0.000592	0.031117	0.206478	0.0	0.0	0.0	1.0	0.033898	0.762712
4	0.003279	0.000296	0.018388	0.423077	0.0	0.0	0.0	1.0	0.338983	0.576271
5	0.029508	0.000296	0.031117	0.238866	0.0	0.0	0.0	1.0	0.203390	0.050847
6	0.004918	0.000296	0.011315	0.560729	0.0	0.0	0.0	1.0	0.186441	0.813559
7	0.004918	0.000296	0.026874	0.807692	0.0	0.0	0.0	1.0	0.016949	0.254237
8	0.021311	0.000296	0.015559	0.803644	0.0	0.0	0.0	1.0	0.338983	0.542373
9	0.019672	0.000592	0.025460	0.354251	0.0	0.0	0.0	1.0	0.084746	0.711864
10	0.029508	0.000296	0.018388	0.238866	0.0	0.0	0.0	1.0	0.271186	0.847458

FIGURE 8. Original click data (Talking Data dataset)

Figure 12 presents the generated click images for the Sweden Ad Click dataset for both fraud and genuine click classes and Figure 9 shows the original sample. By just looking at this image set with the naked eye, anyone can differentiate this dataset from previous datasets.

	timestamp	valid	clientIp	adRelType	forcedAd	testAd	uid	dnt	...	impressionTime
1	0.961720	0.0	0.975430	0.0	0.0	0.0	0.876559	0.0	...	0.961730
2	0.993713	0.0	0.997601	0.0	0.0	0.0	0.741860	0.0	...	0.993713
3	0.953482	0.0	0.983629	0.0	0.0	0.0	0.780370	0.0	...	0.953482
4	0.701098	0.0	0.960417	0.0	0.0	0.0	0.251342	0.0	...	0.701103
5	0.976981	0.0	0.964740	0.0	0.0	0.0	0.990025	0.0	...	0.977004
6	0.761315	0.0	0.959626	0.0	0.0	0.0	0.107213	0.0	...	0.761321
7	0.777114	0.0	0.971607	0.0	0.0	0.0	0.438188	0.0	...	0.777114
8	0.325267	0.0	0.999815	1.0	0.0	0.0	0.002928	0.0	...	0.325266
9	0.917250	0.0	0.981797	0.0	0.0	0.0	0.415433	0.0	...	0.917251
10	0.971950	0.0	0.984868	0.0	0.0	0.0	0.943009	0.0		0.971952

FIGURE 9. Original click data (Sweden Ad click dataset)

B. IMAGE AUGMENTATION

As explained in the methodology section, we employed different image augmentation techniques in C2IDTL. Figure 13 shows a shear range that varies from zero (leftmost) to 0.2 (rightmost) applied to an image. Shearing introduces small image alignment deviations, making a DTL model adapt to small image alignment deviations. Once you apply shearing to your dataset, you will have a similar dataset with smaller deviations. Figure 14 shows the zooming effect from left to right. The leftmost image is with 0 zooming, and the rightmost is with 0.2 max zooming settings. This technique makes sure your model can predict results even with slight scale changes. Figure 15 illustrates the results of different flipping configurations applied. These simple augmentation techniques ensure that the model is not overfitting for the selected sample datasets.

C. QUANTITATIVE AND COMPARATIVE ANALYSIS

In the following section, we will compare and contrast the computed statistical parameters of each DTL model with those of state-of-the-art models as mentioned in the methodology section, we used click datasets as CSV files with state-of-the-art models and generated syntactic image datasets with DTLs. In the last section, we will present the computational based analysis related to the sturdy.

1) STATISTICAL ANALYSIS

The following results are based on 3-fold cross-validation. Here, the dataset was split into three parts, and each model was trained and validated three times. The final metrics were obtained by averaging the values across the folds.

For the FDMA 2012 dataset, Table 8 shows the performances of both states of ML models and DTL models. Out of them, Gradient Boosting Tree outsmarted other ML models with 88.03% average accuracy and other statistical parameters scoring over 87%. Overall, the DTL models repeatedly showed better results compared to the ML models across all the click datasets, proving their robustness and versatility in handling different datasets. These findings indicate their effectiveness and reliability for complex click fraud detection tasks compared to traditional models. The second best ML model was MLP scoring 84.29% accuracy. Both KNN and SVM models showed similar accuracies at 74.33% and 74.39%. This was similar to other parameters as well. The logistic Regression model showed the poorest performance scoring at 63.18%. All the DTL models scored

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

over 89% for all the statistical parameters indicating their capability of handling this dataset. The best model in terms of accuracy was the Xception model which was 92.62%. but the other four models scored over 90% indicating the solid capability of handling this syntactically generated images via proposed C2I approach on FDMA dataset.

With the Talking Data dataset, Table 9 illustrates both Gradient Boosting Tree and KNN overwhelmed other ML models by scoring average accuracies at 95.10% and 96.96%. also, they recorded over 94% percentage for Precision, Recall, F1, and AUC exhibiting their ability to adapt the Talking Data click dataset. The logistic regression model showed the worst capabilities and both SVM and MLP showed comparatively better performances than Logistic Regression. Concerning DTLs, as in the FDMA dataset, all the models scored over 92% of all the parameters. The top performer ResNet50 scored AUC 99.92%, recall 99.61%, precision 98.69%, and accuracy 99.14%. Finally, VGG16 marked its performance as slightly lower compared to ResNet50. Overall all the DTLs surpassed ML models with reference to evaluated statistical parameters.

InceptionV3 model recorded the best accuracy for the Sweden Ad Click dataset by showing 99.73% accuracy in Table 10 along with precision of 99.84%, Recall of 99.62%, and AUC Score of 99.99% outsmarting its performance by surpassing the other models. VGG16 scores the second best accuracy with a recall of 99.96% and a perfect AUC score of 99.99%. Scoring an accuracy of 99.37%, recall of 99.96% and a nearly perfect AUC score of 99.98%, the Resnet50 model also highlights its ability to detect click frauds. Both Gradient Boosting and MLP models showed outstanding performance as ML models compared to other ML models with an Accuracy of 99.36%, Precision of 98.98%, and an impressive Recall of 99.74%. Other ML models also scored perfect scores over 96% with reference to assessed statistical parameters. The Xception model scored 92.62% accuracy as the lowest value compared to the other four DTLs. Besides its lower accuracy, its precision of 99.77% and recall of 99.64% were exceptionally high. Apart from the Xception model's lower accuracy, other statistical parameters indicated that still the traditional ML models cannot match with DTL model's performances.

Figure 16 illustrates a summary of the comparison of statistical metrics for all three models. These charts also clearly visualize that DTL models perform better compared to state-of-the-art models. Especially InceptionV3, DenseNet121, and ResNet50 models show consistent performs for all three datasets. Out of them, the Sweden Ad click dataset showed higher scores indicating overall all the models handled this dataset particularly well. To evaluate statistical significance to compare models, we employed a paired t-test [78]. We conducted tests on 3-fold cross-validation accuracy results for all three datasets. As for the FDMA 2012 dataset, the Xception DTL model showed the best accuracy, and the Gradient Boost ML model scored as the best ML model with $p < 0.01$. Similarly, for the Talking Data dataset, the ResNet50 DTL model demonstrated as statistically significant performance outperforming the best k-nearest neighbor and the best ML model. On the Sweden Ad Click dataset comparison between Inceptionv3 and MLP did not reach the conventional threshold, but still Inception model showed better performance. This reflects the fact that the dataset is well-structured and easily classifiable. We were able to observe this in the ROC curves as well.

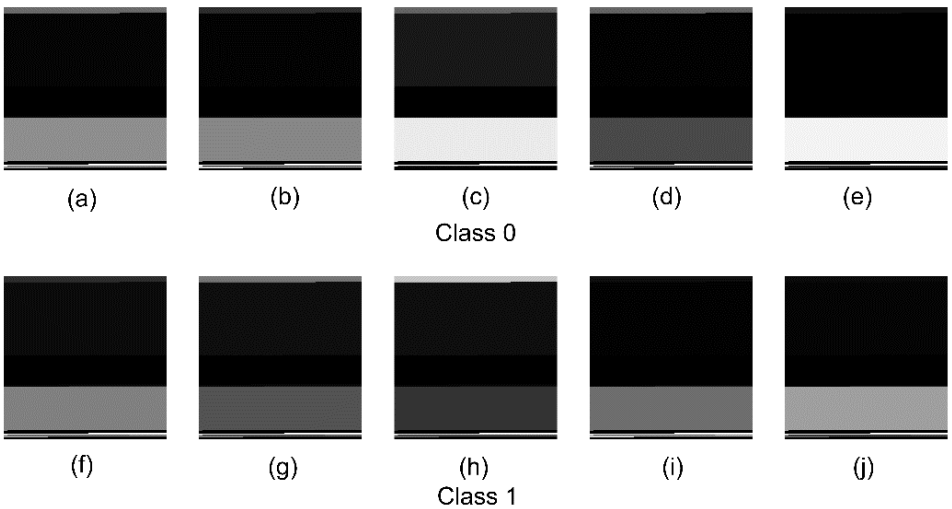


FIGURE 10. **Generated synthetic images with C2I (FDMA 2012 dataset)**

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

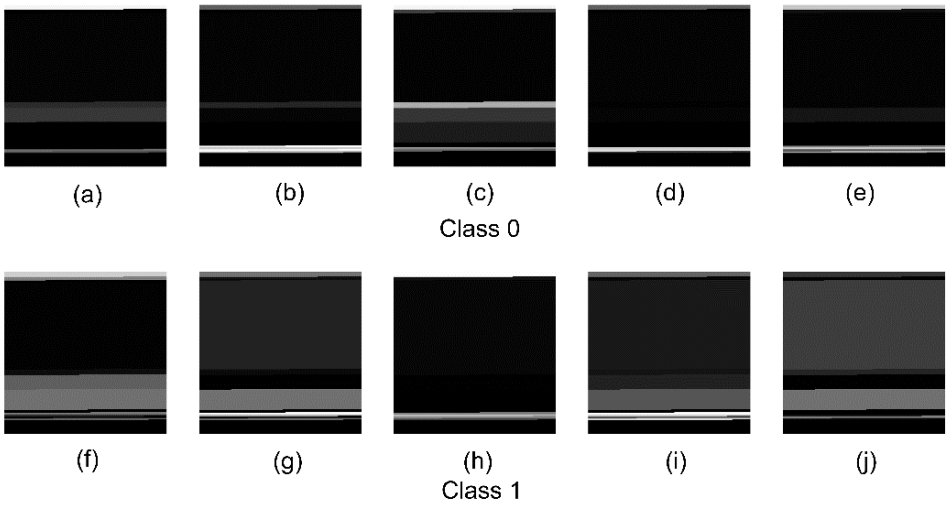


FIGURE 11. **Generated synthetic Images with C2I (Talking dataset)**

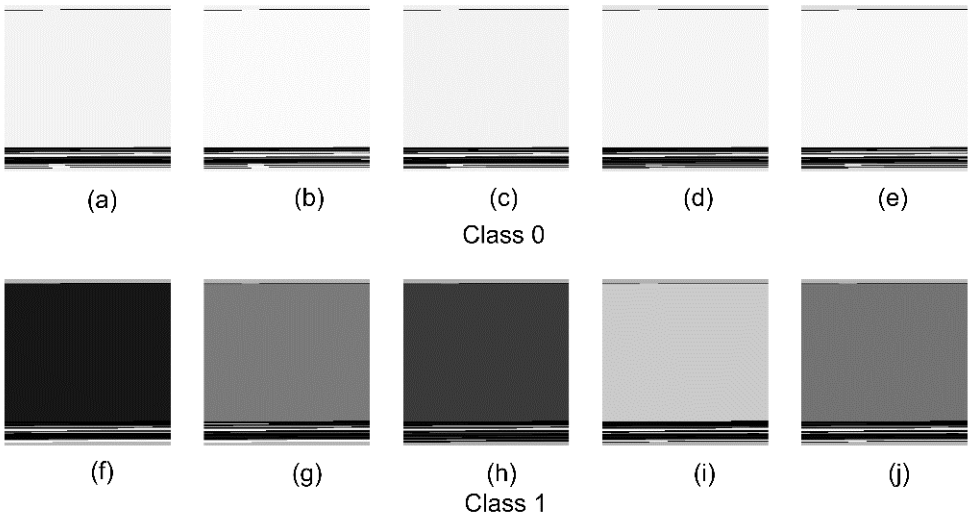


FIGURE 12. **Generated synthetic images with C2I (Sweden dataset)**

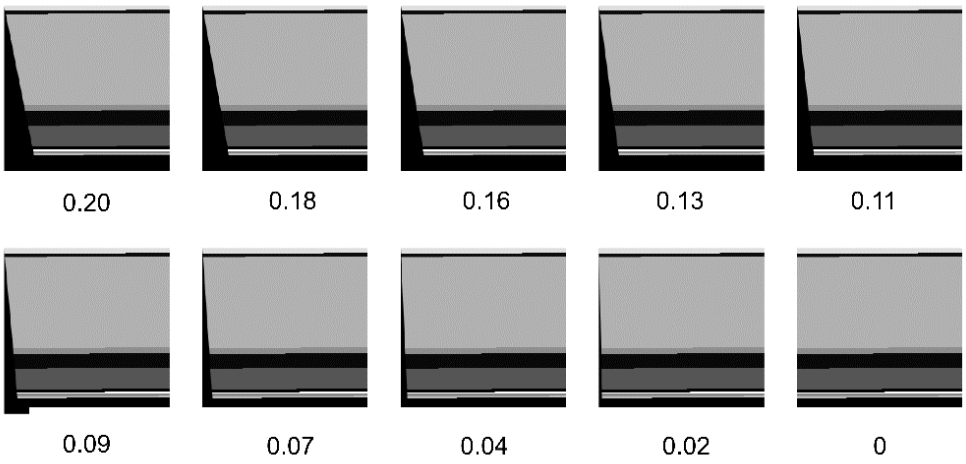


FIGURE 13. **Image augmentation - shearing**

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

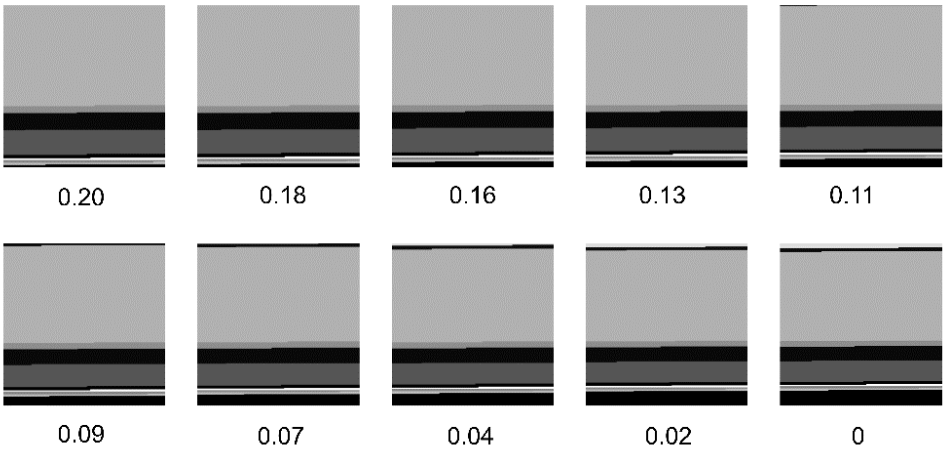


FIGURE 14. **Image augmentation – zooming**

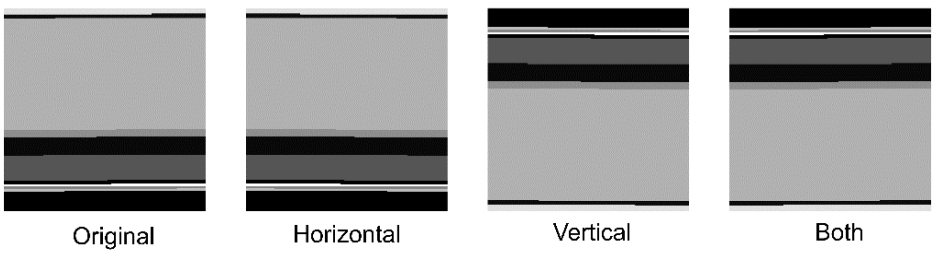


FIGURE 15. **Image augmentation - flipping**

TABLE VIII
COMPARISON OF STATISTICAL METRICS FOR FDMA 2012 DATASET

Model	Accuracy	Precision	Recall	F1 Score	AUC Score
Gradient Boosting Tree	0.8803	0.8964	0.8964	0.8773	0.8802
k-Nearest Neighbors	0.7433	0.7640	0.7017	0.7315	0.7431
Logistic Regression	0.6318	0.6215	0.6685	0.6441	0.6319
Support Vector Machine	0.7439	0.7526	0.7243	0.7381	0.7438
Neural Network (MLP)	0.8429	0.8629	0.8142	0.8378	0.8428
Xception	**0.9262**	0.8991	0.8965	0.8977	0.9645
InceptionV3	0.9011	0.8914	0.9112	0.9012	0.9657
VGG16	0.9035	0.8919	**0.9161**	0.9038	0.9672
ResNet50	0.9147	0.9189	0.9077	**0.9133**	**0.9724**
DenseNet121	0.9118	**0.9290**	0.8899	0.9090	0.9717

TABLE IX
COMPARISON OF STATISTICAL METRICS FOR TALKING DATA DATASET

Model	Accuracy	Precision	Recall	F1 Score	AUC Score
Gradient Boosting Tree	0.9510	0.9846	0.9162	0.9492	0.9509
k-Nearest Neighbors	0.9696	0.9430	**0.9996**	0.9705	0.9696
Logistic Regression	0.7959	0.8205	0.7571	0.7875	0.7959
Support Vector Machine	0.8507	0.8700	0.8244	0.8466	0.8507
Neural Network (MLP)	0.8984	0.8912	0.9092	0.8996	0.8984
Xception	0.9262	0.9654	0.9262	0.9768	0.9974

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

InceptionV3	0.9697	0.9635	0.9764	0.9699	0.9953
VGG16	0.9848	0.9758	0.9940	0.9848	0.9983
ResNet50	**0.9914**	**0.9869**	0.9961	**0.9915**	**0.9992**
DenseNet121	0.9834	0.9758	0.9913	0.9835	0.9983

TABLE X
COMPARISON OF STATISTICAL METRICS FOR SWEDEN AD CLICK DATASET

Model	Accuracy	Precision	Recall	F1 Score	AUC Score
Gradient Boosting Tree	0.9936	0.9898	0.9974	0.9936	0.9994
k-Nearest Neighbors	0.9727	0.9757	0.9696	0.9726	0.9727
Logistic Regression	0.9769	0.9805	0.9732	0.9768	0.9769
Support Vector Machine	0.9833	0.9937	0.9728	0.9831	0.9833
Neural Network (MLP)	0.9964	0.9978	0.9951	0.9964	0.9964
Xception	0.9262	0.9977	0.9964	0.9971	0.9977
InceptionV3	**0.9973**	0.9984	0.9962	**0.9973**	**0.9999**
VGG16	0.9955	**0.9973**	0.9937	0.9955	**0.9999**
ResNet50	0.9937	0.9879	**0.9996**	0.9938	0.9998
DenseNet121	0.9942	0.9974	0.9909	0.9941	0.9998

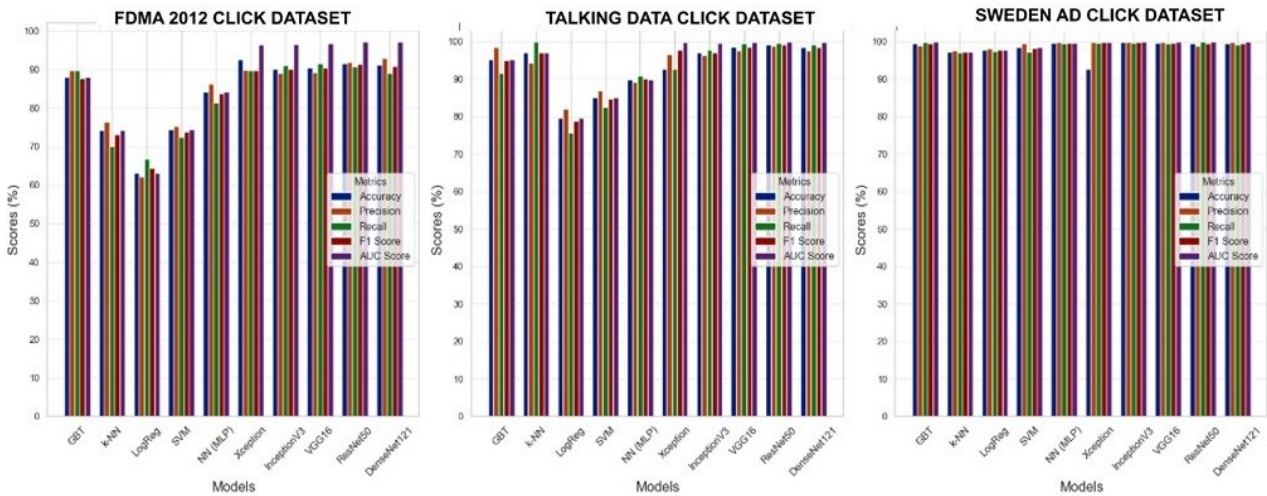


FIGURE 16. **Summary of the model performances**

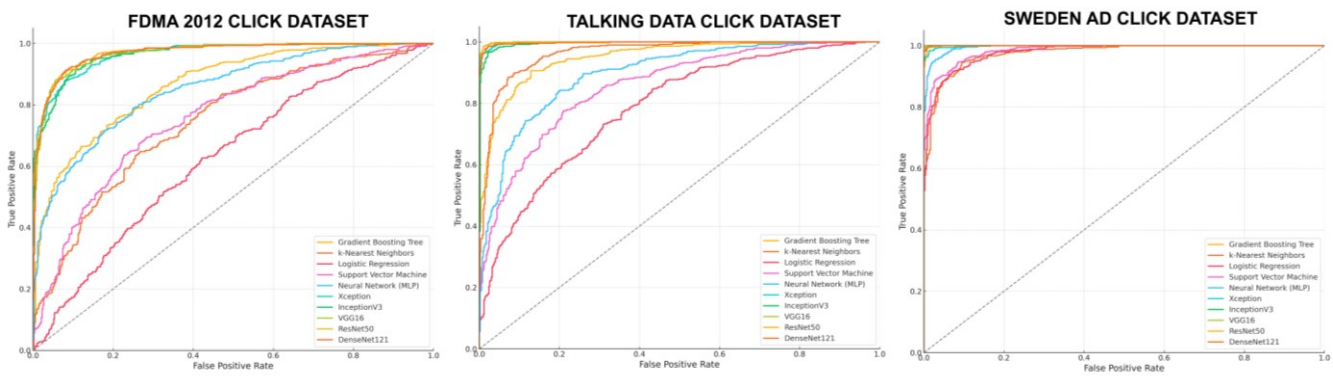


FIGURE 17. **ROC curves for each dataset**

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

Figure 17 presents ROC curves for each dataset for all ten models. These curves illustrate the trade-off between false positive rate (specificity) and true positive rate (sensitivity). These plots represent how well models can differentiate positive and negative classes. The higher the area under the curve, the better the performance. For the FDMA dataset, all 5 DTL models show better performances over 0.9657, indicating their ability in classification fraud clicks, similarly. On the Talking Data dataset, Xception, InceptionV3, VGG16, ResNet50, and DenseNet121 ROC scores over 0.9953 by showing their superiority. Also, GBT and k-nearest performed competitively. The Sweden Ad Click dataset produced the highest scores by exceeding 0.9727 with all the selected ML and DTL models. These values indicate the higher discriminative performance across model types.

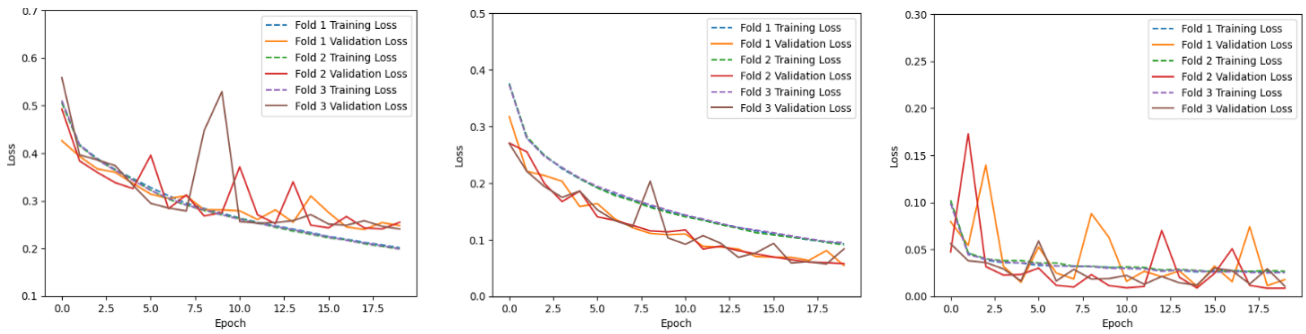
Figure 18 to 22 depicts the training and validation losses for all the datasets with selected DTL models. These models were trained for 20 epochs in each fold. Figure 18 shows the loss curves for the Xception model for all three datasets. For the FDMA 2012, the training loss starts from the 0.45 – 0.50 range and reduces to 0.20 gradually. As for the validation loss, it decreases to 0.25 - 0.28 with some fluctuations. For the Talking dataset, the training loss starts from 0.5 and the validation loss starts from 0.55 to 0.43. After 20 epochs training loss decreases 0.2 and validation loss 0.3. For the Sweden dataset, validation loss starts from 0.28 – 0.33 range and training loss from 0.38 and reduces to 0.05 to 0.09. These loss curves indicate this model performs well over the folds.

As shown in Figure 19, training loss and validation loss for the FDMA dataset starts from around 0.50 and with some fluctuations reduces to the 0.70 – 0.30 range for the InceptionV3 model. For the Talking dataset, both start from 0.30 – 0.40 span and reduce to 0.10 – 0.20 span. Validation loss curves for Sweden curve fluctuate a lot between 0.05 – 0.40 compared to training loss which shows a steady variation. The InceptionV3 model performs well for the first two datasets but not for the Sweden dataset.

Figure 20 depicts loss curves for the VGG16 model. Initial training loss starts from between 0.55 – 0.50 intervals and it gradually lowers below 0.25. Similarly, validation loss also starts from around 0.43 and reduces to below 0.30. For the talking data dataset, both training and validation loss starts between 0.30 – 0.40 upper bounds and scales down to 0.10 – 0.20 lower bound. Losses for the Sweden Ad Click dataset start from 0.10 and decrease to 0.05 with a very low reduction rate. Statistically, VGG16 shows more generalization on all three datasets.

As outlined in Figure 21, All three datasets loss curves of the ResNet50 model can be explained as follows. Training loss for the FDMA dataset starts from 0.45 and uniformly reduces to 0.12. The validation loss initiates to curtail and converge 0.62 – 0.75 span to 0.30. Also for the Talking data dataset, training loss starts just above 0.30 and reduces to below 0.10. The initial validation loss is between 0.23 and 0.27 and reduces to below 0.05 over 20 cycles. Finally, for the Sweden Ad click dataset, training loss begins from 0.05 and decreases to below 0.03 with minimum spikes. However, the validation loss shows a huge variation from 0.45 and scale down with huge ups and downs. As a summary this model also performed well for the first two datasets but not much for the Sweden dataset.

According to Figure 22, Training loss for the FDMA 2012 dataset starts to drop over 0.50 to below 0.20 for the Densnet121 model. In the meantime, validation loss drops to over 0.24. The loss curves for the Talking data dataset for both validation and training start from 0.23 to 0.35 range and decrease below 0.15. But as in the Resnet50 model, both losses show huge variations for the Sweden dataset.

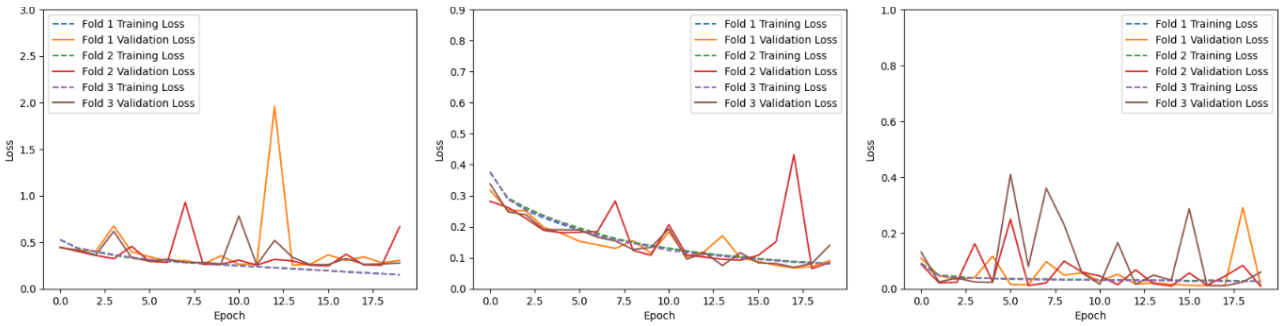


FDMA 2012 Click Dataset **Talking Data Click Dataset** **Sweden Ad Click Dataset**

FIGURE 18. Loss Curves – Xception

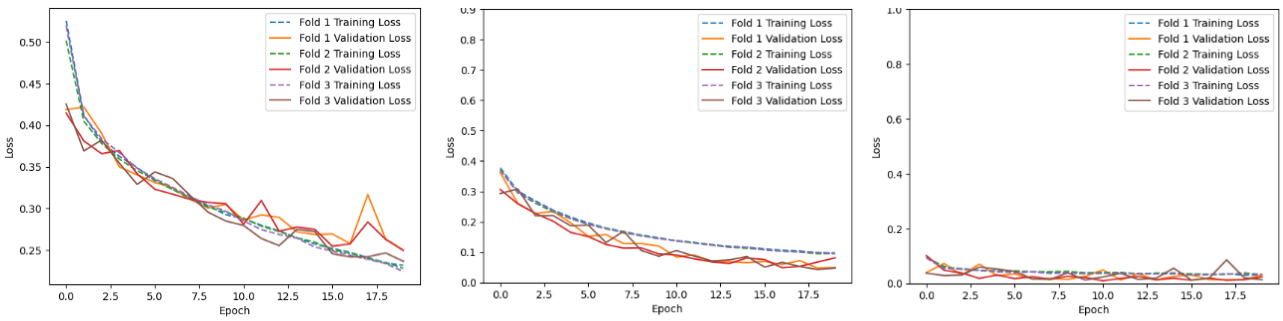
This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187



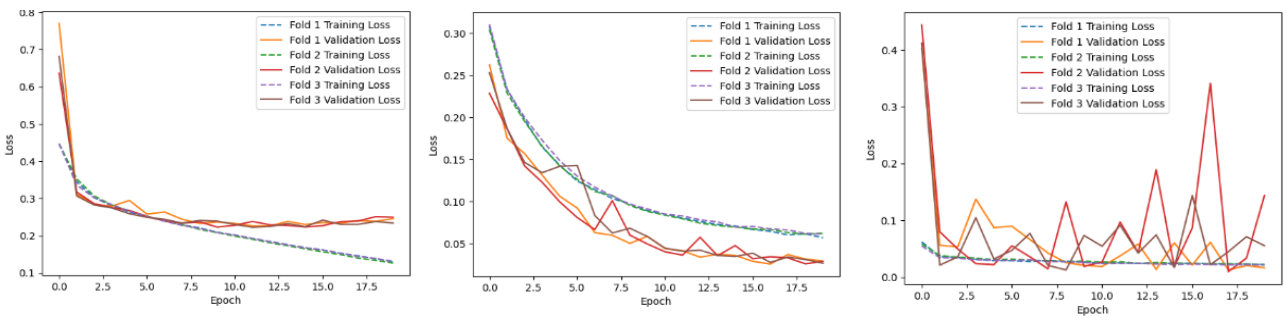
FDMA 2012 Click Dataset | Talking Data Click Dataset | Sweden Ad Click Dataset

FIGURE 19. Loss Curves – InceptionV3



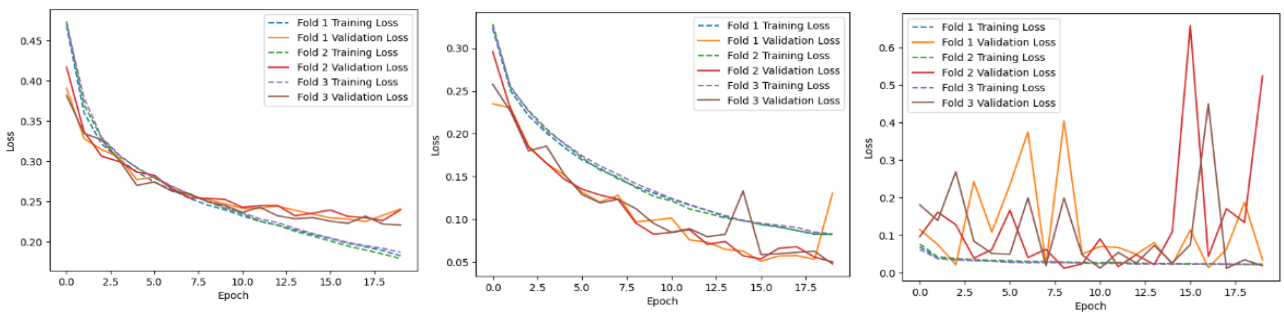
FDMA 2012 Click Dataset | Talking Data Click Dataset | Sweden Ad Click Dataset

FIGURE 20. Loss Curves - VGG16



FDMA 2012 Click Dataset | Talking Data Click Dataset | Sweden Ad Click Dataset

FIGURE 21. Loss Curves – ResNet50



FDMA 2012 Click Dataset | Talking Data Click Dataset | Sweden Ad Click Dataset

FIGURE 22. Loss curves – DesnseNet121

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

2) COMPUTATIONAL ANALYSIS

First, we examined all five DTL models Xception, InceptionV3, VGG16, ResNet50, and DenseNet121 and assessed the average total time for training, the average inference time for predicting results for the validation set and the average total memory usage overall for each model for each dataset. here we have computed the average values for every fold recorded for the comparison in this section. All the computed results for the FDMA 2012 dataset is shown in Table 11 shows how each of above mentioned computational parameters was recorded for all five DTL models. The inference time analysis shows that the InceptionV3 is the fastest model scoring 68.63 seconds while theVGG16 took 76.78 seconds and records it as the second best value. Then the ResNet50 records 84.76 seconds and Xception 82.70 seconds and indicates their internal architecture complexity. Concerning the training time, InceptionV3 again hit a shorter duration of 17708.03 seconds and Resnet50 model 17817.44 seconds by following closely. Then VGG16 model 18135.25seconds and Xception model 18343.85 seconds to train the whole model with the training dataset indicating due to the overhead of the depth wise convolutional operations. As for memory usage, VGG16 has consumed just 2545.78 MB followed by the Xception model of 2761.32 MB. Both Resenet50 and InceptionV3 have been utilized moderately throughout the whole process.

Next Table 12 shows the computational analysis of the Talking Data dataset. Hence the dataset has more samples compared to the other 2 datasets, we can observe that the resulting training time, inference time and memory consumption values are higher and take more resources and time. InceptionV3 model shows a faster inference time of 104.40 seconds and Xception has come close with 104.56 seconds. However, the Resenet50 being the slowest and complex model, scores 114.22 seconds as the inference time. The InceptionV3 model's memory usage is 3000.77 MB indicating that it is the most memory efficient model out of them. InceptionV3 model requires the most memory usage of 3635.63 MB due to its deep architecture, followed by Xception of 3375.82 MB and ResNet50 of 3343.14 MB similar to the previous FDMA 2012 dataset's memory consumption. In terms of the total time, being the fastest ResNet50 spent 20617.88 seconds. Next, the VGG16 with 24692.10 seconds and the InceptionV3 model spent 25316.05 seconds, slightly longer.

For the Sweden Ad Click dataset, the computational analysis is presented in Table 13. Being the smallest dataset and slightly small compared to the FDMA dataset, this dataset shows faster overall performances. InceptionV3 took 58.98 seconds to make predictions on the validation set followed by 64.83 seconds for VGG16. Then the Xception at 69.68 seconds, and theResNet50 at 71.33 seconds which are also less compared to the other datasets. The memory usage trends are slightly different and Xception requires the huge amount of memory at 3104.30 MB. VGG16 is the most memory efficient, just by using only 2761.03 MB as expected. Then followed by InceptionV3 of 3320.37 MB andResNet50 of 2904.44 MB. As for the training time, ResNet50 is the fastest, finishing training in 12588.78 seconds. VGG16 follows with 15304.36 seconds, narrowly outperforming InceptionV3 model 15178.86 seconds and Xception at 15594.26 seconds.

TABLE XI
COMPARISON OF COMPUTATIONAL METRICS FOR FDMA 2012 DATASET

Dataset	Average Inference Time Seconds	Average Memory Usage MB	Average Training Time Seconds
ResNet50	84.76	2884.93	17817.44
Xception	82.70	2761.32	18343.85
InceptionV3	68.63	2889.65	17708.03
VGG16	76.78	2545.78	18135.25
Desenet121	72.07	3111.04	18236.21

TABLE XII
COMPARISON OF COMPUTATIONAL METRICS FOR TALKING DATA DATASET

Dataset	Average Inference Time Seconds	Average Memory Usage MB	Average Training Time Seconds
ResNet50	114.22	3343.14	20617.88
Xception	111.76	3375.82	25885.03
InceptionV3	104.40	3635.63	25316.05
VGG16	104.56	3000.77	24692.10
Desenet121	101.55	3710.75	25600.43

TABLE XIII
COMPARISON OF COMPUTATIONAL METRICS FOR SWEDEN AD CLICK DATASET

Dataset	Average Inference Time Seconds	Average Memory Usage MB	Average Training Time Seconds
ResNet50	71.33	2904.44	12588.78
Xception	69.68	3104.30	15594.26
InceptionV3	58.98	3320.37	15178.86
VGG16	64.83	2761.03	15304.36
Desenet121	65.49	3458.83	15543.07

Figure 23 shows how the Grad-CAM analysis was done on a sample of syntactic images from each dataset. In this analysis, we selected the Xception DTL model and we removed the other layers after the final convolution layer. Sub Figures (a), (c), and(e) are original images from each dataset (FDMA 2012, Talking Data, Sweden Ad Click) and they belonged to class 1. (b), (d), (f) figures are their respective Grad-CAMs images. When the classification results were observed for all three Grad-CAM models, the

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and
content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

predictions were also 1 for these three sample images. In the middle of the FDMA image's Grad-CAM, the yellow and green concentration is high.

That indicates the model mostly focused on the middle area when the decision was given. Its surrounding area is dark green and blue which suggests less contribution to the decision. For the input image generated with the proposed novel C2I approach of the Talking Data sample image, as in the FDMA image, the middle area shows more yellow concentration but comparatively less area. The bottom area is also yellowish towards the bottom center suggesting a strong impact on the decision. The sample image from the Sweden dataset has a more yellow area on the bottom right area of the CAM diagram denoting the importance of this area of the image. But it has a very low yellow concentration in the middle area. All the other areas are dark and suggest a very low contribution to the classification result.

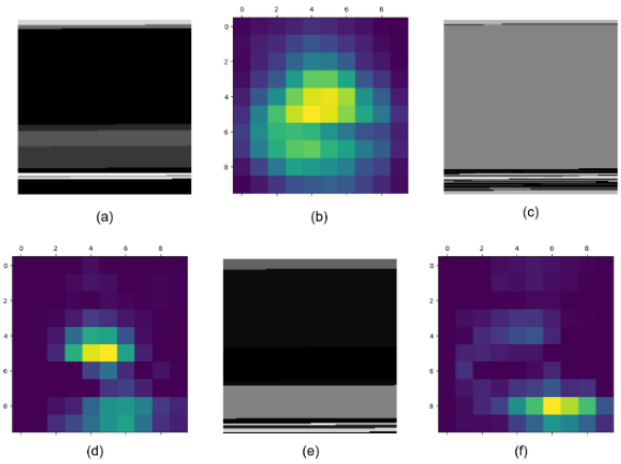


FIGURE 23. **Grad CAM analysis**

V. CONCLUSION AND FUTURE DIRECTION

Our C2IDTL click fraud detection system was proposed to overcome the inability to apply reliable, high performance and re trainable DTL models in the click fraud domain. The converted images were able to preserve the original characteristics of the click dataset (between 2 classes and among datasets) which was visible to visually distinguish. The we evaluated these images with the selected DTL models including ResNet50, Xception, InceptionV3, VGG16 and Desenet121. The statistical and computational analysis showed that these approach consistently produced outstanding performances on all three datasets.

In terms of performance, for the selected sample of the FDMA 2012 dataset, the Xception DTL model showed the best accuracy of 92.62%. Also ResNet50 model showed the best results for the Talking Data dataset which was over 99.14% accuracy and 99.73% accuracy for the Sweden Ad Click dataset by the InceptionV3 model. Also, these DTLs have very slight deviations and suggest that we can use any DTL model out of these selected models depending on the requirement

A key limitation of these DTL models is the higher resource consumption. These models require more GPUs and computational resources for better performance. To address these drawbacks, future studies could examine the use of optimised lightweight alternative DTL models like MobileNetV3, EfficientNet-Lite, or SqueezeNet architectures. These models may offer slightly less fraud detection performance and present a practical tradeoff between computational efficiency and classification accuracy, making them suitable for future investigation.

To improve the statistical performance, future works may explore hybrid approaches that combine ensemble learning algorithms like Random Forests, XGBoost and LightGBM, which may lead to enhanced robustness and generalizability. These ensemble models independently showed better results, and the combined hybrid approach may have the potential to improve results further. Also we can enhance the proposed click to image conversion technique by employing different techniques. This could result in more sophisticated, rich images for click fraud detection. Also, not only for click domain, but also for other domains can apply this technique and generate synthetic images for further studies. This will open another dimension for future studies

REFERENCES

[1] M. Nodirovna and A. Sharif, "E-Commerce Trends: Shaping The Future of Retail," Acad. One, vol. 2, no. 3, pp. 2–2, 2024.

[2] K. van Gelder, "Global e-retail growth rate 2017 to 2027 | Statista." Accessed: Jun. 03, 2024. [Online]. Available: https://www.statista.com/statistics/288487/forecast-of-global-b2c-e-commerce-growth/

[3] M. Mkansi, "E-business adoption costs and strategies for retail micro businesses," Electron. Commer. Res., vol. 22, no. 4, pp. 1153–1193, Dec. 2022, doi: 10.1007/S10660-020-09448-7/FIGURES/3.

[4] X. Liu et al., "Cyber security threats: A never-ending challenge for e-commerce," Front. Psychol., vol. 13, p. 927398, Oct. 2022, doi: 10.3389/FPSYG.2022.927398/BIBTEX.

[5] C. Shi, R. Song, X. Qi, Y. Song, B. Xiao, and S. Lu, "ClickGuard: Exposing Hidden Click Fraud via Mobile Sensor Side-channel Analysis," IEEE Int. Conf. Commun., vol. 2020-June, Jun. 2020, doi: 10.1109/ICC40277.2020.9149420.

[6] M. Kantardzic, C. Walgampaya, B. Wenerstrom, O. Lozitskiy, S. Higgins, and D. King, "Improving click fraud detection by real time data fusion," Proc. 8th IEEE Int. Symp. Signal Process. Inf. Technol. ISSPIT 2008, no. January 2009, pp. 69–74, 2008, doi: 10.1109/ISSPIT.2008.4775655.

[7] I. Aberathne, C. Walgampaya, and U. Rathnayake, "Novel Hidden Markov Scoring Algorithm for Fraudulent Impression Classification in Mobile Advertising," Adv. Intell. Syst. Comput., vol. 1183, no. February 2022, pp. 110–118, 2021, doi: 10.1007/978-981-15-5856-6_10.

[8] F. Zhu, C. Zhang, Z. Zheng, and S. Al Otaibi, "Click Fraud Detection of Online Advertising-LSH Based Tensor Recovery Mechanism," IEEE Trans. Intell. Transp. Syst., vol. 23, no. 7, pp. 9747–9754, Jul. 2022, doi: 10.1109/TITS.2021.3107373.

[9] F. E. Botchey, Z. Qin, K. Hughes-Lartey, and K. E. Ampomah, "Predicting Fraud in Mobile Money Transactions using Machine Learning: The Effects of Sampling Techniques on the Imbalanced Dataset," Inform., vol. 45, no. 7, pp. 45–56, 2021, doi: 10.31449/inf.v45i7.3179.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

[10] C. D. Fernando and C. K. Walgampaya, "Detecting Click Fraud Using an Improved Lenet-5 Convolution Neural Network," Proc. - Int. Res. Conf. Smart Comput. Syst. Eng. SCSE 2023, 2023, doi: 10.1109/SCSE59836.2023.10215038.

[11] M. Jogin, Mohana, M. S. Madhulika, G. D. Divya, R. K. Meghana, and S. Apoorva, "Feature extraction using convolution neural networks (CNN) and deep learning," 2018 3rd IEEE Int. Conf. Recent Trends Electron. Inf. Commun. Technol. RTEICT 2018 - Proc., no. November, pp. 2319–2323, 2018, doi: 10.1109/RTEICT42901.2018.9012507.

[12] M. Long, H. Zhu, J. Wang, and M. I. Jordan, "Deep Transfer Learning with Joint Adaptation Networks," Jul. 17, 2017, PMLR. Accessed: Sep. 23, 2024. [Online]. Available: https://proceedings.mlr.press/v70/long17a.html

[13] M. Hussain, J. J. Bird, and D. R. Faria, "A Study on CNN Transfer Learning for Image Classification," Adv. Intell. Syst. Comput., vol. 840, pp. 191–202, 2019, doi: 10.1007/978-3-319-97982-3_16.

[14] Y. Zhu et al., "Converting tabular data into images for deep learning with convolutional neural networks," Sci. Rep., vol. 11, no. 1, pp. 1–11, 2021, doi: 10.1038/s41598-021-90923-y.

[15] A. Gupta, M. S. Shehata, W. J. Braun, R. E. Feldman, and T. A. Bjarnason, "Automating CT Diagnostic Image Quality Control Using Transfer Learning," Proc. IEEE Int. Conf. Model. Simul. Intell. Comput. MoSICom 2023, pp. 515–520, 2023, doi: 10.1109/MOSICOM59118.2023.10458738.

[16] H. Tang, W. Jing, D. Tang, Z. Yang, X. Yang, and W. Xie, "Global-Local Attention-Aware Zero-Shot Learning for Industrial Fault Diagnosis," IEEE Trans. Instrum. Meas., 2025, doi: 10.1109/TIM.2025.3550233.

[17] X. Huang, "High Resolution Remote Sensing Image Classification Based on Deep Transfer Learning and Multi Feature Network," IEEE Access, vol. 11, pp. 110075–110085, 2023, doi: 10.1109/ACCESS.2023.3320792.

[18] "Buzzcity Mobile Advertisement Dataset | Living Analytics Research Centre." Accessed: May 06, 2024. [Online]. Available: https://larc.smu.edu.sg/buzzcity-mobile-advertisement-dataset

[19] "TalkingData AdTracking Fraud Detection Challenge | Kaggle." Accessed: May 03, 2024. [Online]. Available: https://www.kaggle.com/c/talkingdata-adtracking-fraud-detection

[20] C. M. R. Haider, A. Iqbal, A. H. Rahman, and M. S. Rahman, "An ensemble learning based approach for impression fraud detection in mobile advertising," J. Netw. Comput. Appl., vol. 112, pp. 126–141, Jun. 2018, doi: 10.1016/J.JNCA.2018.02.021.

[21] K. Wang, G. Xu, C. Wang, and X. He, "A Hybrid Abnormal Advertising Traffic Detection Method," Proc. - 2017 IEEE Int. Conf. Big Knowledge, ICBK 2017, pp. 236–241, Aug. 2017, doi: 10.1109/ICBK.2017.50.

[22] D. Sisodia and D. S. Sisodia, "Feature space transformation of user-clicks and deep transfer learning framework for fraudulent publisher detection in online advertising," Appl. Soft Comput., vol. 125, p. 109142, Aug. 2022, doi: 10.1016/J.ASOC.2022.109142.

[23] E.-A. MINASTIREANU and G. MESNITA, "Light GBM Machine Learning Algorithm to Online Click Fraud Detection," J. Inf. Assur. Cybersecurity, no. April, pp. 1–12, 2019, doi: 10.5171/2019.263928.

[24] A. Dash and S. Pal, "Auto-Detection of Click-Frauds using Machine Learning Auto-Detection of Click-Frauds using Machine Learning," no. September, 2020.

[25] J. Wang, C. Wu, S. Ji, Q. Gu, and Z. Li, "Fraud Detection via Coding Nominal Attributes," ACM Int. Conf. Proceeding Ser., pp. 42–45, Aug. 2017, doi: 10.1145/3145511.3145514.

[26] N. P. Gohil and A. D. Meniya, "Click Ad Fraud Detection Using XGBoost Gradient Boosting Algorithm," pp. 67–81, 2021, doi: 10.1007/978-3-030-76776-1_5.

[27] T. G.S., S. Dheeshjith, S. S. Iyengar, N. R. Sunitha, and P. Badrinath, "A hybrid and effective learning approach for Click Fraud detection," Mach. Learn. with Appl., vol. 3, p. 100016, Mar. 2021, doi: 10.1016/J.MLWA.2020.100016.

[28] R. Dekou, S. Savo, S. Kufeld, D. Francesca, and R. Kawase, "Machine Learning Methods for Detecting Fraud in Online Marketplaces," CEUR Workshop Proc., vol. 3052, no. July, pp. 1–8, 2021.

[29] B. Mikkili and S. Sodagudi, "Advertisement Click Fraud Detection Using Machine Learning Algorithms," Smart Innov. Syst. Technol., vol. 282, pp. 353–362, 2022, doi: 10.1007/978-981-16-9669-5_32.

[30] B. Kirkwood, M. Vanamala, and N. Seliya, "Click Fraud Detection of Online Advertising Using Machine Learning Algorithms," IEEE Int. Conf. Electro Inf. Technol., pp. 586–590, 2024, doi: 10.1109/EIT60633.2024.10609899.

[31] V. B. Mahesh, K. V. S. Chandra, L. S. P. Babu, V. A. Sowjanya, and M. Mohammed, "Clicking Fraud Detection for online advertising using Machine Learning," 2024 4th Int. Conf. Intell. Technol. CONIT 2024, 2024, doi: 10.1109/CONIT61985.2024.10627189.

[32] D. Sisodia and D. S. Sisodia, "A transfer learning framework towards identifying behavioral changes of fraudulent publishers in pay-per-click model of online advertising for click fraud detection," Expert Syst. Appl., vol. 232, p. 120922, Dec. 2023, doi: 10.1016/J.ESWA.2023.120922.

[33] R. Mouawi, M. Awad, A. Chehab, I. H. El Hajj, and A. Kayssi, "Towards a Machine Learning Approach for Detecting Click Fraud in Mobile Advertizing," Proc. 2018 13th Int. Conf. Innov. Inf. Technol. IIT 2018, pp. 88–92, Jul. 2018, doi: 10.1109/INNOVATIONS.2018.8605973.

[34] M. Gabryel, M. M. Scherer, Ł. Sułkowski, and R. Damaševičius, "Decision Making Support System for Managing Advertisers by Ad Fraud Detection," J. Artif. Intell. Soft Comput. Res., vol. 11, no. 4, pp. 331–339, 2021, doi: 10.2478/jaiscr-2021-0020.

[35] H. Chari, S. Aswale, V. N. Pawar, P. Shetgaonkar, and K. M. Chaman Kumar, "Advertisement Click Fraud Detection Using Machine Learning Techniques," Proc. Int. Conf. Technol. Adv. Innov. ICTAI 2021, pp. 109–114, 2021, doi: 10.1109/ICTAI53825.2021.9673199.

[36] M. Renström and T. Holmsten, "Fraud Detection on Unlabeled Data with Unsupervised Machine Learning," p. 62, 2018, [Online]. Available: http://kth.diva-portal.org/smash/get/diva2:1217521/FULLTEXT01.pdf

[37] A. Batool and Y. C. Byun, "An Ensemble Architecture Based on Deep Learning Model for Click Fraud Detection in Pay-Per-Click Advertisement Campaign," IEEE Access, vol. 10, no. October, pp. 113410–113426, 2022, doi: 10.1109/ACCESS.2022.3211528.

[38] S. P. Sreekala, B. Rajnarayanan, V. Vijayakumar, M. Mathi, S. Revathy, and S. Jeyalakahmi, "UNLEASHING DEEPNETS TO COMBAT AD CLICK FRAUD AND SAFEGUARDING ONLINE ADVERTISING," 2023 IEEE Int. Conf. Res. Methodol. Knowl. Manag. Artif. Intell. Telecommun. Eng. RMKMATE 2023, 2023, doi: 10.1109/RMKMATE59243.2023.10369226.

[39] B. Joy and V. R. Deepthi, "A Tensor Based Approach for Click Fraud Detection on Online Advertising Using BiLSTM and Attention based CNN," Int. Conf. Self Sustain. Artif. Intell. Syst. ICSSAS 2023 - Proc., pp. 669–674, 2023, doi: 10.1109/ICSSAS57918.2023.10331862.

[40] R. A. Alzahrani, M. Aljabri, and R. M. A. Mohammad, "Ad Click Fraud Detection Using Machine Learning and Deep Learning Algorithms," IEEE Access, 2025, doi: 10.1109/ACCESS.2025.3532200.

[41] B. Dai, L. Wang, X. Zhao, Y. Guo, and M. S. Obaidat, "GemmaWithLoRA: A New Approach to Click Fraud Detection," Proc. 2024 IEEE Int. Conf. Commun. Comput. Cybersecurity Informatics, CCCI 2024, 2024, doi: 10.1109/CCCI61916.2024.10736481.

[42] J. Hu, T. Li, Y. Zhuang, S. Huang, and S. Dong, "GFD: A Weighted Heterogeneous Graph Embedding Based Approach for Fraud Detection in Mobile Advertising," Secur. Commun. Networks, vol. 2020, no. 1, p. 8810817, Jan. 2020, doi: 10.1155/2020/8810817.

[43] C. Chen et al., "Deep Learning on Computational-Resource-Limited Platforms: A Survey," Mob. Inf. Syst., vol. 2020, no. 1, p. 8454327, Jan. 2020, doi: 10.1155/2020/8454327.

[44] A. Sharma, E. Vans, D. Shigemizu, K. A. Boroevich, and T. Tsunoda, "DeepInsight: A methodology to transform a non-image data to an image for convolution neural network architecture," Sci. Rep., vol. 9, no. 1, pp. 1–7, 2019, doi: 10.1038/s41598-019-47765-6.

[45] A. Sharma, A. Lysenko, K. A. Boroevich, E. Vans, and T. Tsunoda, "DeepFeature: feature selection in nonimage data using convolutional neural network," Brief. Bioinform., vol. 22, no. 6, pp. 1–12, 2021, doi: 10.1093/bib/bbab297.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

[46] F. Chollet, "Xception: Deep learning with depthwise separable convolutions," Proc. - 30th IEEE Conf. Comput. Vis. Pattern Recognition, CVPR 2017, vol. 2017-January, pp. 1800–1807, Nov. 2017, doi: 10.1109/CVPR.2017.195.

[47] X. Xia, C. Xu, and B. Nan, "Inception-v3 for flower classification," 2017 2nd Int. Conf. Image, Vis. Comput. ICIVC 2017, pp. 783–787, Jul. 2017, doi: 10.1109/ICIVC.2017.7984661.

[48] K. Simonyan and A. Zisserman, "Very Deep Convolutional Networks for Large-Scale Image Recognition," 3rd Int. Conf. Learn. Represent. ICLR 2015 - Conf. Track Proc., Sep. 2014, Accessed: Oct. 06, 2024. [Online]. Available: https://arxiv.org/abs/1409.1556v6

[49] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," Proc. IEEE Comput. Soc. Conf. Comput. Vis. Pattern Recognit., vol. 2016-December, pp. 770–778, Dec. 2016, doi: 10.1109/CVPR.2016.90.

[50] G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger, "Densely Connected Convolutional Networks," Proc. - 30th IEEE Conf. Comput. Vis. Pattern Recognition, CVPR 2017, vol. 2017-January, pp. 2261–2269, Aug. 2016, doi: 10.1109/CVPR.2017.243.

[51] M. Taneja, K. Garg, A. Purwar, and S. Sharma, "Prediction of click frauds in mobile advertising," 2015 8th Int. Conf. Contemp. Comput. IC3 2015, pp. 162–166, Dec. 2015, doi: 10.1109/IC3.2015.7346672.

[52] K. S. Perera, B. Neupane, M. A. Faisal, Z. Aung, and W. L. Woon, "A Novel Ensemble Learning-Based Approach for Click Fraud Detection in Mobile Advertising," Lect. Notes Comput. Sci. (including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics), vol. 8284 LNAI, pp. 370–382, 2013, doi: 10.1007/978-3-319-03844-5_38.

[53] R. Oentaryo et al., Detecting click fraud in online advertising: A data mining approach, vol. 15. 2014.

[54] K. S. Perera, B. Neupane, M. A. Faisal, Z. Aung, and W. L. Woon, "A novel ensemble learning-based approach for click fraud detection in mobile advertising," Lect. Notes Comput. Sci. (including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics), vol. 8284 LNAI, pp. 370–382, 2013, doi: 10.1007/978-3-319-03844-5_38.

[55] C. D. Fernando, P. T. Weerasinghe, and C. K. Walgampaya, "Heart Disease Risk Identification using Machine Learning Techniques for a Highly Imbalanced Dataset: a Comparative Study," KDU J. Multidiscip. Stud., vol. 4, no. 2, pp. 43–55, Dec. 2022, doi: 10.4038/KJMS.V4I2.50.

[56] L. B. V. de Amorim, G. D. C. Cavalcanti, and R. M. O. Cruz, "The choice of scaling technique matters for classification performance," Appl. Soft Comput., vol. 133, pp. 1–37, 2023, doi: 10.1016/j.asoc.2022.109924.

[57] J. Quackenbush, "Microarray data normalization and transformation," vol. 32, no. december, pp. 496–501, 2002, doi: 10.1038/ng1032.

[58] B. Nithya, R. Saranya, M. Joseph, R. Josphineleela, A. Thirunagalingam, and A. K. Kodali, "Convolutional-Neural Network Bases Speaker Emotion Detection Based on Human-Computer Interaction," 1st Int. Conf. Adv. Comput. Sci. Electr. Electron. Commun. Technol. CE2CT 2025, pp. 738–743, 2025, doi: 10.1109/CE2CT64011.2025.10939767.

[59] P. H. Do, T. D. Le, T. D. Dinh, and V. D. Pham, "Classifying IoT Botnet Attacks with Kolmogorov-Arnold Networks: A Comparative Analysis of Architectural Variations," IEEE Access, 2025, doi: 10.1109/ACCESS.2025.3528940.

[60] S. K. Banerjee, P. Chakraborty, and S. Nath, "Hybrid Transfer Learning based Pomegranate Fruit Disease Identification," Int. Conf. Big Data Anal. Bioinformatics, DABCon 2024, 2024, doi: 10.1109/DABCON63472.2024.10919383.

[61] P. Yadav et al., "Investigation and Empirical Analysis of Transfer Learning for Industrial IoT Networks," IEEE Access, 2024, doi: 10.1109/ACCESS.2024.3499741.

[62] B. H. Menze et al., "A comparison of random forest and its Gini importance with standard chemometric methods for the feature selection and classification of spectral data," BMC Bioinformatics, vol. 10, no. 1, pp. 1–16, Jul. 2009, doi: 10.1186/1471-2105-10-213/TABLES/4.

[63] J. Deng, W. Dong, R. Socher, L. J. Li, K. Li, and L. Fei-Fei, "ImageNet: A Large-Scale Hierarchical Image Database," 2009 IEEE Conf. Comput. Vis. Pattern Recognition, CVPR 2009, pp. 248–255, 2009, doi: 10.1109/CVPR.2009.5206848.

[64] C. Szegedy, S. Ioffe, V. Vanhoucke, and A. A. Alemi, "the Impact of Residual Connections on Learning," pp. 4278–4284.

[65] L. Taylor and G. Nitschke, "Improving Deep Learning with Generic Data Augmentation," Proc. 2018 IEEE Symp. Ser. Comput. Intell. SSCI 2018, pp. 1542–1547, Jul. 2018, doi: 10.1109/SSCI.2018.8628742.

[66] S. H. S. Basha, S. R. Dubey, V. Pulabaigari, and S. Mukherjee, "Impact of fully connected layers on performance of convolutional neural networks for image classification," Neurocomputing, vol. 378, pp. 112–119, 2020, doi: 10.1016/j.neucom.2019.10.008.

[67] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov, "Dropout: A Simple Way to Prevent Neural Networks from Overfitting," J. Mach. Learn. Res., vol. 15, no. 56, pp. 1929–1958, 2014, Accessed: Oct. 03, 2024. [Online]. Available: http://jmlr.org/papers/v15/srivastava14a.html

[68] D. P. Kingma and J. L. Ba, "Adam: A Method for Stochastic Optimization," 3rd Int. Conf. Learn. Represent. ICLR 2015 - Conf. Track Proc., Dec. 2014, Accessed: Oct. 03, 2024. [Online]. Available: https://arxiv.org/abs/1412.6980v9

[69] A. A. Rusu et al., "Progressive Neural Networks," no. June, 2016, doi: 10.48550/arXiv.1606.04671.

[70] D. Yu, H. Wang, P. Chen, and Z. Wei, "Mixed Pooling for Convolutional Neural Networks," Lect. Notes Comput. Sci. (including Subser. Lect. Notes Artif. Intell. Lect. Notes Bioinformatics), vol. 8818, pp. 364–375, 2014, doi: 10.1007/978-3-319-11740-9_34.

[71] S. Sharma and R. Mehra, "Conventional Machine Learning and Deep Learning Approach for Multi-Classification of Breast Cancer Histopathology Images-a Comparative Insight," J. Digit. Imaging, vol. 33, no. 3, pp. 632–654, Jun. 2020, doi: 10.1007/S10278-019-00307-Y.

[72] T. Fushiki, "Estimation of prediction error by using K-fold cross-validation," Stat. Comput., vol. 21, no. 2, pp. 137–146, Apr. 2011, doi: 10.1007/S11222-009-9153-8/METRICS.

[73] S. Akila, R. Keerthiga, V. Vijayan, and S. Prabha, "Satellite Image Assisted Lake and River Detection using ResNet: An Analysis," 2025 Int. Conf. Electron. Renew. Syst., pp. 1617–1621, Feb. 2025, doi: 10.1109/ICEARS64219.2025.10940143.

[74] P. K. Sahu and T. Fatma, "Optimized Breast Cancer Classification Using PCA-LASSO Feature Selection and Ensemble Learning Strategies with Optuna Optimization," IEEE Access, 2025, doi: 10.1109/ACCESS.2025.3539746.

[75] S. Arunmozhi, P. Arunagiri, and S. Prabha, "Normal/Cataract Detection in Fundus Image Using Individual and Fused ResNet Features," 2024 Int. Conf. Syst. Comput. Autom. Networking, ICSCAN 2024, 2024, doi: 10.1109/ICSCAN62807.2024.10894554.

[76] A. Heller, A. Liu, L. C. Chang, G. Cohen, E. C. Jones, and L. Y. Hsu, "Performance Evaluation of Multi-Contrast Dixon MRI and CT for Abdominal Fat and Muscle Segmentation Using a UNet CNN," Proc. - 2024 IEEE Int. Conf. Big Data, BigData 2024, pp. 8665–8667, 2024, doi: 10.1109/BIGDATA62323.2024.10825381.

[77] V. A. S and M. M. C. M, "Automatic Detection of Healthy/Disease Paddy Plant Image using Deep Learning," 2025 Int. Conf. Electron. Renew. Syst., pp. 1958–1962, Feb. 2025, doi: 10.1109/ICEARS64219.2025.10940340.

[78] I. Kilic and G. Aydin, "Traffic Sign Detection and Recognition Using TensorFlow' s Object Detection API with A New Benchmark Dataset," 2020 Int. Conf. Electr. Eng. ICEE 2020, Sep. 2020, doi: 10.1109/ICEE49691.2020.9249914.

[79] E. Lindholm, J. Nickolls, S. Oberman, and J. Montrym, "NVIDIA Tesla: A unified graphics and computing architecture," IEEE Micro, vol. 28, no. 2, pp. 39–55, Mar. 2008, doi: 10.1109/MM.2008.31.

[80] R. R. Selvaraju, M. Cogswell, A. Das, R. Vedantam, D. Parikh, and D. Batra, "Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization," Proc. IEEE Int. Conf. Comput. Vis., vol. 2017-October, pp. 618–626, Dec. 2017, doi: 10.1109/ICCV.2017.74.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/

This article has been accepted for publication in IEEE Access. This is the author's version which has not been fully edited and content may change prior to final publication. Citation information: DOI 10.1109/ACCESS.2025.3571187

CHANNA FERNANDO is currently pursuing an MPhil degree in Engineering Mathematics at the University of Peradeniya, Sri Lanka. He earned his Bachelor of Science in Engineering, graduating with First Class Honours from the Department of Electronic and Telecommunication Engineering, Faculty of Engineering, University of Moratuwa, in 2020. Following his graduation, he joined the Faculty of Engineering at the University of Peradeniya in 2021 as a temporary lecturer at the Computing Center. In 2024, he was promoted to the position of Lecturer (Probationary) in the same faculty. His research interests lie at the intersection of artificial intelligence, machine learning (ML), deep learning (DL), and software development. He is particularly focused on fraud detection using advanced machine learning and deep learning models, with a strong interest in applying these techniques to domains such as financial technology and online security. His work also explores the Internet of Things (IoT), investigating how intelligent systems can be used to improve connectivity, automation, and efficiency in a range of applications.

CHAMILA WALGAMPAYA is a professor in computer engineering at the Faculty of Engineering. In December 2001, Chamila joined the Department of Engineering Mathematics at the University of Peradeniya, marking the commencement of his influential career in academia. He was awarded the Fulbright scholarship in August 2004, leading him to pursue an MS in Computer Engineering at the University of Louisville, Kentucky, U.S.A. He successfully completed his master's degree in May 2006, presenting a thesis titled "Cost-benefit Analysis in Multiple Time Series Prediction," which demonstrated his innovative approach to optimizing sensor selection in distributed energy-producing plant networks. Chamila's academic journey culminated in the attainment of a Ph.D. from the School of Engineering at the University of Louisville in August 2011. His doctoral research, encapsulated in a dissertation titled "Click Fraud: how to spot it? how to stop it?" focused on the development of a pioneering click fraud detection and prevention system for Internet advertising. In addition to his roles as a professor and researcher, Chamila Walgampaya serves as the Director of the computing center at the Faculty of Engineering since March 2022.

JANAKA ALAWATUGODA received his B.Sc. degree (Hons.) in computer science from the University of Peradeniya, Sri Lanka, and his Ph.D. from Queensland University of Technology, Australia. Currently, he serves as an Associate Researcher–Assistant Professor within the Research and Innovation Centers Division at Rabdan Academy, United Arab Emirates. He holds positions as an Adjunct Research Fellow at the Institute for Integrated and Intelligent Systems, Griffith University, Australia; a Visiting Lecturer at the Postgraduate Institute of Science, University of Peradeniya; and a Reviewer for zbMATH Open Database, FIZ Karlsruhe, Germany. He is recognized as a Fellow of the British Computer Society (BCS) and a Senior Member of the Association for Computing Machinery (ACM). He has been honored with the Commendations for Faculty Research Excellence from the Faculty of Engineering, University of Peradeniya, in 2018 (for the years 2016–2017), 2020 (for the years 2018–2019), and 2022 (for the years 2020–2021). In addition, he has been appreciated from Rabdan Academy, United Arab Emirates, in 2023, for consistently achieving the highest levels of student satisfaction and demonstrating unwavering dedication.

This work is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 License.
For more information, see https://creativecommons.org/licenses/by-nc-nd/4.0/